

A PROJECT REPORT ON
RESUME SCREENING SYSTEM USING NLP

Submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE & ENGINEERING

Submitted By

B. SAILU
K. AKHILA
MD.MALLIKA
G.S.V.N. RAJU

22MQ1A0561
22MQ1A0520
22MQ1A0531
22MQ1A0568

Under the Esteemed Supervision of

B.INDRA DEVI
Assistant Professor



... Empowering Minds

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SRI VASAVI INSTITUTE OF ENGINEERING & TECHNOLOGY

Accredited by NAAC with 'A' Grade & NBA

Approved by AICTE-New Delhi & Affiliated to JNTU Kakinada

An ISO9001:2015 Certified Institute

Nandamuru, Pedana(M)-521369, Krishna Dist., Andhra Pradesh.

2022-2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SRI VASAVI INSTITUTE OF ENGINEERING & TECHNOLOGY

Accredited by NAAC with 'A' Grade & NBA

Approved by AICTE-New Delhi & Affiliated to JNTU Kakinada

An ISO 9001:2015 Certified Institute

Nandamuru, Pedana(M)-521369, Krishna Dist, Andhra Pradesh.

2022-2026



CERTIFICATE

This is to certify that the project report entitled "RESUME SCREENING SYSTEM USING NLP " is a Bonafide work carried out by B.Sailu (22MQ1A0561), K.Akhila (22MQ1A0520), MD.Mallika (22MQ1A0531),G.S.V.N.Raju(22MQ1A0568).Under the guidance and supervision in partial fulfillment of the requirements for the award of degree of B. Tech in Computer Science and Engineering from Jawaharlal Nehru Technological University, Kakinada .The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma.

PROJECT GUIDE

B. INDRA DEVI

M.Tech., (Ph.D).

Assistant Professor

HEAD OF THE DEPARTMENT

Dr.G.SYAM PRASAD

B.Tech., M.Tech., Ph.D.

Professor & HOD

EXTERNAL EXAMINER

DECLARATION

We certify that,

- The work contained in this report is original and has been done by us under the guidance of our supervisor.
- The work has not been submitted to any other institute for any degree or diploma. We have followed the guidelines provided by the institute in preparing the report.
- We confirm the norms and guidelines given in the Ethical code of Conduct of the institute.
- Whenever we have used materials (data, theoretical analysis, figures and text) from the sources, we have given due credit to them by references.
- Further, we have taken permission from the copyright owners of the sources whenever necessary.

Signature(s) of the student(s):

B.Sailu
K.Akhila
MD.Mallika
G.S.V.N.Raju

22MQ1A0561
22MQ1A0520
22MQ1A0531
22MQ1A0568

ACKNOWLEDGEMENT

We take great pleasure to express our deep sense of gratitude to our project guide **B.INDRA DEVI, Assistant Professor** for her valuable guidance during the course of our project work.

We would like to thank **Dr.G.SYAM PRASAD, Professor & Head of the Department of Computer Science and Engineering**, for his encouragement.

We would like to express our heart-felt thanks to **Dr.B.R.S.REDDY**, Principal, **Sri Vasavi Institute of Engineering & Technology**, Nandamuru for providing all the facilities for our project. Our utmost thanks to all the faculty members and Non-Teaching Staff of the Department of Computer Science and Engineering for their support throughout our project work.

Our Family Members and Friends receive our deepest gratitude and love for their support throughout our academic year.

Project Associates

B.Sailu

K.Akhila

MD.Mallika

G.S.V.N.Raju

22MQ1A0561

22MQ1A0520

22MQ1A0531

22MQ1A0568

ABSTRACT

Recruitment and talent acquisition are critical focuses for organizations and enterprises.

Understanding the reasons behind inefficient hiring — where suitable candidates are overlooked or unqualified applicants are shortlisted due to manual screening limitations — is essential for building a productive and skilled workforce, which contributes to long-term organizational growth. This project emphasizes the use of Exploratory Data Analysis (EDA) and Machine Learning (ML) models to automate and optimize the resume screening process effectively.

By analyzing historical recruitment data, including candidate skills, educational background, work experience, certifications, technical competencies, and job description requirements, the system aims to identify key indicators of candidate suitability. Natural Language Processing (NLP) techniques are used to process and extract meaningful information from unstructured resume text. Advanced machine learning algorithms such as Logistic Regression, Decision Trees, Random Forest, Gradient Boosting (XG Boost), Support Vector Machines (SVM), and Neural Networks are employed to build predictive classification and ranking models. These models leverage textual patterns, keyword relevance, and semantic relationships within the data to forecast which candidates best match specific job roles.

Additionally, EDA is applied to derive insights into resume trends by visualizing and interpreting skill frequency, experience distribution, keyword importance, and feature correlations. This step improves feature engineering and model performance, ensuring that the most meaningful variables are used for candidate evaluation. Model performance is evaluated using metrics such as Accuracy, Precision, Recall, and F1-Score to ensure reliable, unbiased, and actionable predictions.

By implementing this intelligent resume screening system, organizations can take proactive measures to improve recruitment efficiency. Strategies may include automated candidate ranking, skill-gap analysis, personalized job recommendations, and data-driven hiring decisions. Moreover, companies can gain valuable insights into industry skill demands and recruitment patterns, supporting more informed and strategic workforce planning.

This project delivers a scalable and data-driven solution for resume screening using NLP and Machine Learning, enabling improved hiring accuracy, reduced manual effort, enhanced recruitment speed, and overall organizational growth.

Index Terms: Natural Language Processing (NLP), Machine Learning, Exploratory Data Analysis (EDA), Resume Screening, Text Classification, XG Boost, Recruitment Analytics, Talent Acquisition System

LIST OF CONTENTS

S.NO	CHAPTER CONTENT	PAGE NO
1.	CHAPTER 1	1-3
	Introduction	2
	1.1 Scope	3
	1.2 Purpose	3
	1.3 Keywords	3
2.	CHAPTER 2	4-10
	Objective ofthe project	5
	2.1 Existing methods	5-7
	2.2 Proposed methods	8
	2.3 Modules Description	9-10
3.	CHAPTER 3	11-13
	Literature survey	12-13
4.	CHAPTER 4	14-16
	Feasibility Report	15
	4.1 Economic Feasibility	15
	4.2 Technical Feasibility	16
	4.3 Social Feasibility	16
5.	CHAPTER 5	17-22
	Systemrequirement specification	18
	5.1 Functional Requirements	19
	5.1.1 Hardware Requirements	19
	5.1.2 Software Requirements	19
	5.2 Non-Functional Requirements	19
	5.3 SDLC Methodologies	20-21
	5.4 System Architecture	22
6.	CHAPTER 6	23-30

	System design	24
	6.1 UML Diagrams	25
	6.2 Use Case Diagram	26
	6.3 Class Diagram	27
	6.4 Sequence Diagram	28
	6.5 Component Diagram	29
	6.6 Object Diagram	30
7.	CHAPTER 7	31-51
	Technology description & implementation	32-51
8.	CHAPTER 8	52-56
	Source code	53-56
9.	CHAPTER 9	57-63
	Systemtesting	58-63
	9.1 Test Cases	64
10.	CHAPTER 10	65-67
	Input and output design	66-67
11.	CHAPTER 11	68-80
	Results	69-80
12.	CHAPTER 12	81-82
	Conclusion & future work	82
13.	CHAPTER 13	83-84
	References	84

LIST OF FIGURES

S.No.	Figures	Page No
1	PER-PDR Relationship	21-22
2	Use Case Diagram	26
3	Class Diagram	27
4	Sequence Diagram	28
5	Component Diagram	29
6	Object Diagram	30
7	Python Version Download and Installation	37-44
8	Software Testing and its types	58
9	Dataset	69
10	Running the terminal to start the Python Server	71
11	Results(Output Screens)	73-80

LIST OF ABBREVIATIONS

ABBREVIATION	EXPLANATION
TF-IDF	Term Frequency Inverse Document Frequency
SVM	Support Vector Machine
DT	DecisionTree
LR	Logistic Regression
ML	Machine Learning
EDA	Exploratory Data Analysis

LIST OF TABLES

TABLE No.	TABLES	Page.no
9.1	Test cases	64
10	Input	66
11	Result Analysis and Comparison Table	80

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

Resume screening is one of the most critical processes in recruitment and talent acquisition. In today's competitive job market, organizations receive a large number of applications for every job opening. Maintaining an efficient and accurate screening process is essential for selecting the right candidates and ensuring long-term organizational success. Companies face challenges not only in attracting qualified applicants but also in evaluating resumes effectively within limited time. Manual resume screening often leads to delays, inconsistencies, and human bias. Identifying and automating this process using intelligent systems helps organizations improve hiring quality and avoid losing potential talent.

With advancements in data analytics, Natural Language Processing (NLP), and Machine Learning (ML), organizations now have access to powerful tools for analyzing resume data. Every resume contains valuable information such as skills, educational background, work experience, certifications, and project details. By leveraging NLP techniques, unstructured textual data in resumes can be processed and transformed into meaningful structured information. Machine learning algorithms can then identify patterns and correlations that indicate candidate suitability for specific job roles. These insights enable recruiters to make proactive, faster, and data-driven hiring decisions.

Importance of Efficient Resume Screening

Efficient resume screening has a direct impact on an organization's productivity and recruitment success. Studies show that selecting candidates who closely match job requirements improves overall workforce performance and organizational growth. In contrast, ineffective screening may result in poor hiring decisions, increased recruitment costs, and reduced operational efficiency.

By understanding which candidates are most suitable for a role, organizations can take timely steps to shortlist and interview the right applicants. Techniques such as keyword matching, skill-based ranking, and automated evaluation systems enhance recruitment efficiency. Maintaining an intelligent resume screening system not only reduces time-to-hire but also improves fairness, minimizes bias, enhances candidate experience, and strengthens long-term organizational performance.

1.1 Scope of the Project

The project leverages diverse recruitment and resume data, including:

- **Candidate Demographics:** Educational background, graduation year, and location.
- **Resume Information:** Skills, certifications, technical expertise, and work experience.
- **Job Description Data:** Required skills, qualifications, experience level, and role specifications.
- **Application Details:** Project work, internships, achievements, and previous job roles.

While focused on automated resume screening, the methodology is applicable across various industries and recruitment platforms, including corporate hiring, campus recruitment, job portals, and consultancy-based talent acquisition systems.

1.2 Purpose of the Project

The purpose of this project is to develop a **Machine Learning and NLP-based system** that automates the resume screening process. By analyzing textual and structured data from resumes, the system identifies candidates who best match specific job requirements. This enables organizations to shortlist suitable applicants efficiently, reduce manual effort, minimize bias, and improve overall hiring accuracy and recruitment speed.

1.3 Keywords

Machine Learning, Natural Language Processing (NLP), Exploratory Data Analysis (EDA),

Resume Screening, Text Classification, XGBoost, Recruitment System

CHAPTER 2

OBJECTIVE OF THE PROJECT

2. OBJECTIVE OF THE PROJECT

2.1 EXISTING METHOD

Predicting bank customer churn has been a challenge for financial institutions for decades. Traditionally, banks relied on manual techniques, basic statistical models, and rule-based systems for identifying customers at risk of leaving. However, these methods often lacked accuracy, scalability, and the ability to predict churn in real-time. In recent years, advancements in machine learning and data analytics have led to the development of more sophisticated models. This section covers the commonly used existing methods for bank customer churn prediction.

1. Keyword Matching Algorithms

Keyword matching is one of the most basic and widely used techniques in traditional resume screening systems. This method scans resumes for specific keywords that match the job description, such as required skills, programming languages, certifications, or qualifications.

For example, if a job requires “Python, Machine Learning, SQL,” the system searches for these exact terms in candidate resumes. Resumes containing more matching keywords are ranked higher.

Advantages:

- Simple and fast implementation
- Easy to understand and interpret
- Works well for structured job requirements

Disadvantages:

- Cannot understand context or synonyms
- May miss qualified candidates using different terminology
- High false positives due to keyword stuffing

2. Rule-Based Filtering

Rule-based filtering operates on predefined conditions set by recruiters or HR professionals.

These rules automatically accept or reject resumes based on specific criteria.

Examples:

- Minimum 2 years of experience required
- Mandatory degree qualification

Advantages:

- Clear and transparent decision logic
- Easy to implement
- Suitable for basic screening tasks

Disadvantages:

- Inflexible and rigid
- Cannot adapt to changing job requirements
- Does not handle complex patterns in resume data

3. Boolean Search Algorithm

Boolean search uses logical operators such as **AND, OR, NOT** to filter resumes based on keyword combinations.

Advantages:

- More precise than simple keyword matching
- Flexible search combinations
- Easy to apply in recruitment databases

Disadvantages:

- Still depends on exact keyword presence
- Does not capture semantic meaning
- Requires manual query construction

4. Simple Text Parsing Techniques

Text parsing techniques extract structured information from resumes, such as name, email, skills, education, and experience. These systems convert unstructured resume text into structured data fields

For example:

- Extract years of experience
- Identify skill lists

Advantages:

- Converts resumes into structured format

Disadvantages:

- Limited contextual understanding
- Errors in parsing different resume formats
- Cannot evaluate candidate suitability accurately

5. Manual Scoring Methods

Manual scoring involves recruiters reviewing resumes and assigning scores based on criteria such as skill relevance, experience level, and qualification match.

For example:

- Skill match – 40%
- Experience – 30%
- Education – 20%
- Certifications – 10%

Advantages:

- Human judgment and contextual understanding
- Flexible evaluation criteria

Disadvantages:

- Time-consuming
- Prone to bias
- Not scalable for large applicant volumes

Conclusion

However, the limitations of traditional resume screening methods show the need for a more advanced and adaptive solution. Basic techniques like keyword matching and rule-based filtering cannot understand context or complex resume data.

By using **Natural Language Processing (NLP)**, deep learning, and advanced feature engineering, the system can better analyze

skills, experience, and job requirements. This improves candidate-job matching accuracy, reduces manual effort, and enhances recruitment efficiency.

Thus, an intelligent Resume Screening System using NLP enables faster, fair, and data-driven hiring decisions.

2.2 PROPOSED METHOD :

The proposed method aims to develop a robust **Machine Learning and NLP-based system** for automated resume screening and candidate-job matching. Unlike traditional keyword-based or manual screening approaches, this system leverages advanced algorithms, comprehensive text analysis, and intelligent prediction techniques. Through the integration of data preprocessing, feature engineering, text vectorization (such as TF-IDF or word embeddings), and model optimization, the solution is designed to improve screening accuracy while providing meaningful insights for recruiters.

The system identifies how well a candidate's skills, experience, and qualifications align with job requirements using data analysis and machine learning models. It analyzes resume content, job descriptions, and candidate profiles to detect matching patterns. Classification models such as **XGBoost**, Random Forest, and Support Vector Machines are explored for predictive modeling and candidate suitability scoring. Exploratory Data Analysis (EDA) is used for feature extraction, understanding skill distribution, and improving model performance.

This intelligent model aims to enhance hiring accuracy, reduce false positives, and streamline the recruitment process

Advantages:

- **Data-Driven Decision Making** – Enables objective and evidence-based candidate selection.
- **Improved Recruitment Efficiency** – Reduces manual screening time and effort.
- **Better Candidate-Job Matching** – Enhances hiring quality and organizational productivity.

2.3 MODULE DESCRIPTION:

1. Data Collection and Integration

The first step involves collecting resumes and job-related data from multiple sources:

- **Resume Data:** PDF/DOC resumes uploaded by candidates.
- **Job Descriptions:** Required skills, experience, qualifications, and responsibilities
- **Candidate Metadata:** Name, email, education, certifications, years of experience, and technical skills.

The system integrates resume data with job requirement data to enable intelligent matching.

2. Data Preprocessing and Cleaning

To ensure accurate model performance, raw resume data must be cleaned and structured:

- **Text Extraction:** Extracting text from PDF/DOC files using NLP libraries.
- **Text Cleaning:** Removing special characters, stopwords, punctuation, and irrelevant symbols.
- **Tokenization:** Splitting text into meaningful words or phrases
- **Lemmatization/Stemming:** Converting words into their base forms.
- **Handling Missing Information:** Managing incomplete resumes or missing fields.

This step converts unstructured resume text into structured and analyzable format.

3. Feature Engineering and Vectorization

After preprocessing, important features are extracted:

- **Skill Extraction:** Identifying technical and soft skills using NLP techniques.
- **Keyword Matching:** Comparing resume keywords with job description keywords.
- **TF-IDF / Word Embeddings:** Converting text into numerical vectors.
- **Experience Calculation:** Extracting years of experience from resume text.

These features help the machine learning model understand resume relevance.

4. Model Selection and Development

Machine learning models are developed to classify and rank resumes based on job suitability.

- **Logistic Regression:** For binary classification (Selected / Not Selected).
- **Support Vector Machine (SVM):** For text classification tasks.
- **Random Forest:** For robust classification performance.
- **Gradient Boosting (XGBoost, LightGBM):** High-performance ensemble models.

Multiple models are trained and compared to select the best-performing model.

5. Model Training and Evaluation

A proper training pipeline ensures model accuracy and reliability:

- **Data Splitting:** Dividing dataset into training, validation, and test sets.

Evaluation Metrics:

- Accuracy
- Precision

- Recall
- F1-Score

6. Resume Scoring and Ranking System

The system assigns a Resume Score based on:

- ✓ Skill Match Percentage
- ✓ Experience Relevance
- ✓ Education Qualification Match
- ✓ Keyword Similarity Score

Candidates are ranked automatically from highest to lowest relevance.

7. Real-Time Prediction and Deployment

Once optimized, the model is deployed for real-time resume screening.

- **API Integration:** RESTful APIs developed using Flask or FastAPI.
- **Database Integration:** Storing resumes and results in PostgreSQL.
- **Admin Dashboard:** HR can view ranked candidates and filtering options.
- **Monitoring & Maintenance:** Model re-training with new hiring data.

The system continuously evaluates incoming resumes and provides instant shortlisting recommendations.

Conclusion:

The **Resume Screening System Using NLP** leverages Data Science and Machine Learning techniques to automate candidate shortlisting. By combining Natural Language Processing, feature engineering, and predictive modeling, the system reduces manual screening effort, improves hiring accuracy, and speeds up recruitment processes.

This intelligent and adaptive solution ensures fair, efficient, and scalable recruitment for modern organizations.

CHAPTER 3

LITERATURE SURRVEY

3.LITERATURE SURVEY

Automated resume screening is emerging as a significant application of **Natural Language Processing (NLP)** and Machine Learning in the recruitment domain. As organizations receive thousands of applications for a single job posting, predictive analytics and intelligent text processing play a vital role in identifying suitable candidates efficiently. This literature review explores various methodologies, algorithms, and challenges related to resume screening by examining research studies, academic contributions, and industry practices in recruitment analytics.

1. Understanding Resume Screening in Recruitment

Resume screening refers to the process of evaluating candidate resumes to determine their suitability for a specific job role. Unlike traditional hiring methods that rely heavily on manual review, automated screening systems analyze structured and unstructured resume data to rank or classify candidates.

According to recent research in recruitment analytics, understanding patterns in skills, qualifications, and experience is essential for improving hiring decisions. Earlier approaches relied on keyword matching and manual heuristics, which were limited in contextual understanding. With the advancement of NLP and ML techniques, organizations now adopt data-driven systems to enhance candidate-job matching accuracy.

2. Machine Learning Approaches for Resume Screening

Various machine learning algorithms have been applied to automate resume classification and candidate ranking.

Supervised Learning Algorithms

- **Logistic Regression (LR):** commonly used for binary classification tasks such as suitable vs. unsuitable candidate prediction. It is valued for its interpretability and simplicity.
- **Decision Trees and Random Forests:** Effective in handling non-linear relationships and identifying important features such as skill relevance and experience level. Ensemble methods like **XGBoost** have demonstrated improved performance in resume classification tasks.

- **Support Vector Machines (SVM):** Widely used for high-dimensional text classification problems, especially when resumes are represented using TF-IDF vector

3. Feature Engineering and Selection

Feature engineering is essential for improving resume screening accuracy. Common techniques include:

- a. **Text Vectorization:** TF-IDF and Word Embeddings to convert textual data into numerical format.
- b. **Normalization and Scaling:** Improves model convergence and performance.

Model interpretability tools such as **SHAP** and feature importance analysis are used to understand which skills or qualifications most influence predictions.

4. Evaluation Metrics for Inactivity Prediction Models

- a. **Accuracy:** Measures overall correctness of predictions
 - b. **Precision and Recall:** Important for reducing false positives and false negatives in candidate selection.
 - c. **F1-Score:** Balances precision and recall in imbalanced datasets.
 - d. **ROC-AUC:** Evaluates classification performance across different threshold values.
- These metrics ensure reliable and fair candidate evaluation.

5. Real-World Challenges and Solutions

- a. **Data Imbalance:** Suitable candidates may represent a smaller portion of applicants; techniques like SMOTE can address imbalance.
- b. **Bias and Fairness:** Ethical AI practices are required to avoid discrimination in automated hiring systems.

Conclusion

The reviewed literature indicates that Machine Learning and NLP provide a scalable and efficient framework for automated resume screening. By leveraging supervised learning algorithms, performing effective EDA, and applying proper evaluation metrics, organizations can enhance candidate-job matching accuracy. With advancements in deep learning and explainable AI, resume screening systems continue to improve in terms of accuracy, transparency, and real-time applicability, supporting intelligent and data-driven recruitment

CHAPTER 4

SYSTEM ANALYSIS

4. SYSTEM ANALYSIS

4.1 FEASIBILITY STUDY

During the system analysis phase, the feasibility of the proposed **Resume Screening System using NLP** is evaluated. The main objective of this system is to automate resume shortlisting by analyzing candidate resumes using Natural Language Processing (NLP) and Machine Learning (ML) techniques

The system helps HR teams reduce manual effort, improve screening accuracy, and speed up recruitment decisions.

Three major areas considered during the feasibility analysis are:

- **ECONOMICAL FEASIBILITY**
- **TECHNICAL FEASIBILITY**
- **SOCIAL FEASIBILITY**

4.1.1 ECONOMICAL FEASIBILITY

- The project is cost-effective because it uses open-source tools like Python and ML libraries, which do not require licensing fees.
- Development costs mainly involve system design, model training, and interface development.
- Maintenance costs are minimal as the models can be retrained periodically using updated resume datasets.
- By automating resume screening, the organization saves:
 - Time spent on manual resume filtering
 - HR operational costs
 - Cost of delayed hiring
- Faster and more accurate hiring leads to better workforce quality, indirectly improving organizational productivity and revenue.

Therefore, the long-term return on investment (ROI) justifies the implementation cost, making the system economically feasible.

4.1.2 TECHNICAL FEASIBILITY

- The Resume Screening System is built using widely adopted Data Science technologies such as **Python**, **scikit-learn**, and **NLTK** for NLP processing and classification.
 - The system processes resumes in structured formats (PDF/DOCX converted to text) and extracts key information like skills, experience, education, and keywords using NLP techniques such as tokenization, stop-word removal, and TF-IDF vectorization.
 - Machine Learning models such as Logistic Regression, Naïve Bayes, or Random Forest can be implemented without requiring high-end hardware. The system can run efficiently on standard systems with moderate RAM and processing capacity.
 - The platform can be integrated with existing HR Management Systems (HRMS) through APIs, making deployment technically feasible with minimal infrastructure changes
- Thus, the project is technically feasible as it uses lightweight, open-source, and scalable technologies.

4.1.3 SOCIAL FEASIBILITY

- The system is designed primarily for HR managers, recruiters, and hiring teams. It reduces manual screening time and helps them focus on strategic hiring decisions.
- The interface is user-friendly and provides resume scores, ranking, and matching percentage against job descriptions.
- Proper training sessions and documentation will be provided to ensure smooth adoption.
- The system promotes fairness by using standardized evaluation criteria, reducing human bias in the initial screening process.
- It assists decision-making but does not replace human judgment, ensuring better acceptance among HR professionals.

Hence, the system is socially acceptable and beneficial to recruitment teams

CHAPTER 5
SYSTEM REQUIREMENTS
AND
SPECIFICATIONS

5. SYSTEM REQUIREMENTS AND SPECIFICATIONS

A Software Requirements Specification (SRS) is a detailed document that describes the complete behavior, features, and constraints of the system to be developed. The **Resume Screening System using NLP** is designed to automate the resume evaluation process using Natural Language Processing (NLP) and Machine Learning techniques to assist recruiters in identifying the most suitable candidates efficiently.

The SRS includes functional requirements, non-functional requirements, use cases, and system constraints to ensure clarity between business stakeholders and the development team.

analyzing the business needs of their clients and stakeholders to help identify business problems and propose solutions. Within the systems development life cycle domain, typically performs a liaison function between the business side of an enterprise and the information technology department or external service providers.

Projects are subject to three sorts of requirements:

► **Business Requirements**

Business requirements define the organizational goals to be achieved:

- Automate manual resume screening process.
- Reduce recruitment time and operational costs.
- Improve candidate-job matching accuracy.
- Minimize human bias in initial screening.
- Provide data-driven hiring insights

► **Product Requirements**

Product requirements describe system features:

- Upload resumes (PDF/DOCX).
- Extract candidate details (skills, experience, education).
- Perform keyword matching with job descriptions.
- Generate Resume Score / Matching Percentage.
- Rank candidates based on suitability.
- Admin dashboard for viewing shortlisted profiles.

► **Process Requirements**

Process requirements define development constraints:

- Follow Agile Iterative Development Model.
- Use standardized coding practices.
- Maintain documentation for model training and evaluation.
- Ensure data privacy compliance.
- Limit development cost using open-source technologies.

5.1 FUNCTIONAL REQUIREMENTS:

5.1.1 HARDWARE REQUIREMENTS

- intel i5 or above processor.
- Minimum 8 GB RAM
- Storage: 256 GB SSD

5.1.2 SOFTWARE REQUIREMENTS

- Operating System: Windows/Linux/Mac
- Python: 3.8 or higher
- Frame work: node.js

5.2 NONFUNCTIONAL REQUIREMENTS:

- Provide fast and accurate resume analysis
- Ensure high accuracy in skill extraction and matching
- Handle large numbers of resumes efficiently
- Easy to use and user-friendly interface
- Secure handling of candidate resume data

5.3 SDLC METHODOLOGIES:

SDLC MODEL

The Software Development Life Cycle (SDLC) for the **Resume Screening System using NLP** follows an **Iterative Development Model**, suitable for small to medium-scale data-driven applications. Since the project involves resume parsing, Natural Language Processing (NLP), machine learning model training, dashboard development, an iterative approach ensures continuous improvement and controlled feature expansion.

In this project, system components are developed through a series of structured iterations. The first iteration focuses on implementing basic functionalities such as resume upload and text extraction. Subsequent iterations enhance the system by adding advanced NLP-based job matching, resume

scoring, ranking mechanisms, and performance optimization.

Six Stages of SDLC (Applied to Resume Screening System)

The six stages of the SDLC are designed to build upon one another. Each stage produces outputs that serve as inputs for the next stage, ensuring traceability and systematic development.

1. Requirement Analysis

- Identify HR recruitment challenges.
- Define system objectives such as automated resume shortlisting.
- Gather business and functional requirements.
- Define NLP and ML expectations

2. System Design

- Design system architecture (Frontend, Backend, ML Module).
- Define database schema for storing resumes and scores.
- Select technologies such as Python, NLP libraries, and web framework.
- Design UI for HR dashboard.

3. Implementation (Development)

- Develop resume upload module.
- Implement text preprocessing (tokenization, stop-word removal).
- Apply NLP techniques for keyword extraction.
- Implement similarity scoring using ML algorithms.
- Develop dashboard for candidate ranking

4. Testing

- Model accuracy testing.
- Performance testing with bulk resumes.
- Fix bugs identified during validation.
- Unit testing of modules.

5. Deployment

- Deploy system locally or on cloud.
- Integrate with HR systems if required.

- Ensure secure access and authentication.

6. Maintenance

- Monitor system performance.
- Retrain ML models with updated datasets.
- Improve matching accuracy.
- Add enhancements based on HR feedback.

Roles and Responsibilities of PDR AND PER

The iterative lifecycle specifies two critical roles that act together to clearly communicate project issues and concepts between the end-user community and the development team.

Primary End-user Representative (PER)

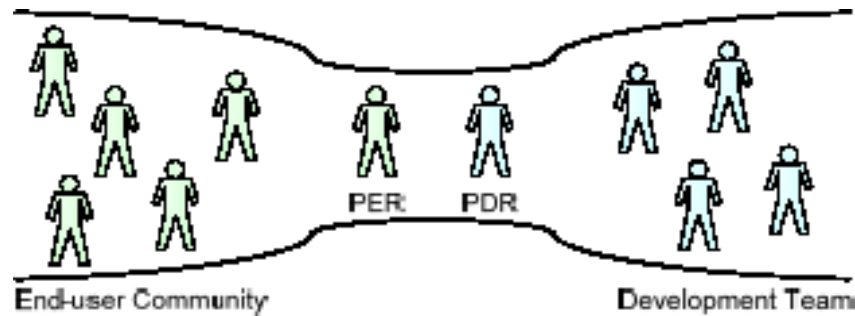
The PER is a person who acts as the primary point of contact and principal approver for the enduser community. The PER is also responsible for ensuring that appropriate subject matter experts conduct enduser reviews in a timely manner.

PER-PDR Relationship

The PER and PDR are the brain trust for the development effort. The PER has the skills and domain knowledge necessary to understand the issues associated with the business processes to the supported by the application and has a close working relationship with the other members of the end- user community.

The PDR has the same advantages regarding the application development process and the other members of the development team together, they act as the concentration points for knowledge about the application to be developed. The objective of this approach is to create the close relationship that is characteristic of a software project with one developer and one end-user in essence, this approach the “pair programming” concept from Agile methodologies and extends it to the end-user community. While it is difficult to create close relationships between the diverse members of an end- user community and a software development team, it is much simpler to create a close relationship between the lead representatives for each group. When multiple end-users are placed into relationship with multiple members of a development team, communication between the two groups degrades as the number of participants grows.

In this model, members of end-user community may communicate with members of the development team as needed, but it is the responsibility of all participants to keep the PER and PDR apprised of the communications for example, this allows the PER and PDR to resolve conflicts that arise when two different end-users communicate different requirements for the same application feature to different members of the development team.



5.4 System Architecture:

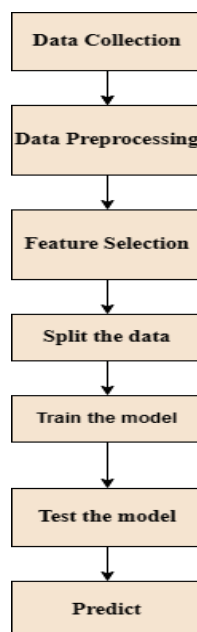


Fig. System Architecture

CHAPTER 6

SYSTEM DESIGN

6. SYSTEM DESIGN

6.1 UML DIAGRAMS

UML (Unified Modeling Language) is a standardized modeling language used to visualize, specify, construct, and document the components of a software system. In the **Resume Screening System using NLP**, UML diagrams are used to represent system structure, data flow, user interactions, and processing modules in a clear and graphical manner.

UML helps in designing object-oriented and data-driven applications by providing a visual understanding of how different components such as Resume Parser, NLP Engine, Scoring Module, and Admin Dashboard interact with each other.

The UML standard is managed by the **Object Management Group**, which defines modeling standards used across software engineering industries.

The Resume Screening System uses UML diagrams to:

- Model system architecture
- Represent HR user interactions
- Define data processing workflows
- Visualize object relationships
- Document system behavior

GOALS:

The Primary goals in the design of the UML are as follows:

► **Provide Ready-to-Use Visual Modeling Language**

The system is designed in a modular way:

- Resume Parsing Module
- NLP Matching Module
- Scoring & Ranking Module
- Admin Dashboar

Future extensions such as AI-based candidate recommendation or interview prediction models can be added without redesigning the entire system.

► **Provide Formal Basis for Understanding**

UML diagrams provide a structured blueprint of:

- System architecture
- Data flow
- Component interaction
- Model training workflow

This helps both technical and non-technical stakeholders understand system functionality.

► **Support Higher-Level Development Concepts**

The system design supports:

- Modular architecture
- Scalable ML models
- Pattern-based ranking mechanisms

This ensures maintainability and scalability in future versions.

UML Diagrams Used in Resume Screening System

1. Use Case Diagram

- Shows interaction between HR/Admin and the system.
- Use cases: Upload Resume, Enter Job Description, View Scores, Export Results.

2. Class Diagram

- Represents system classes and their relationships.
- Example: Resume, Candidate, NLP Processor, MatchingEngine

3. Collaboration Diagram – system components interact to process resumes, compute similarity with job requirements, and generate ranked candidate results.

4. Sequence Diagram

- Describes interaction between modules during resume processing

► **Encourage Use of Object-Oriented Tools:**

The Resume Screening System is built using object-oriented principles:

- Classes for Resume
- Classes for Candidate
- Classes for Job Description
- Classes for Matching Eng

USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

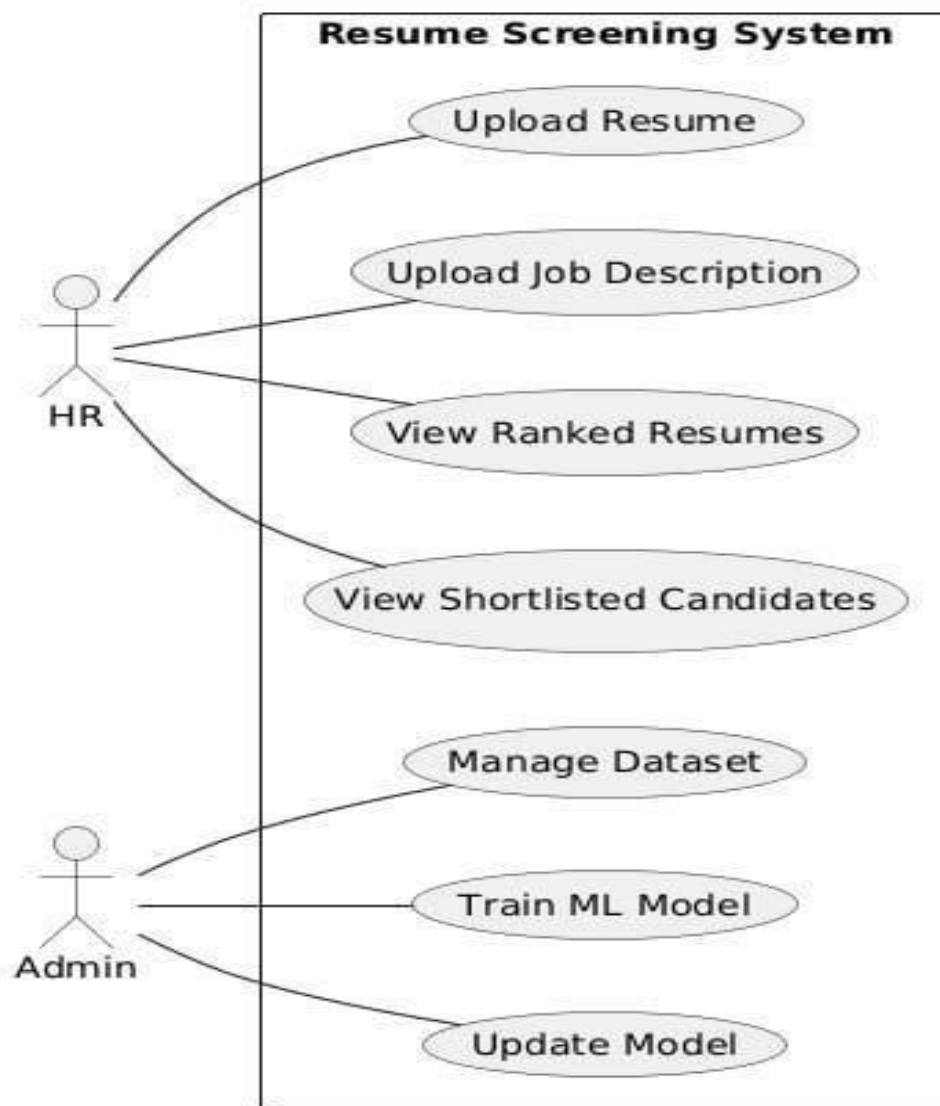


Fig: Use Case Diagram

CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

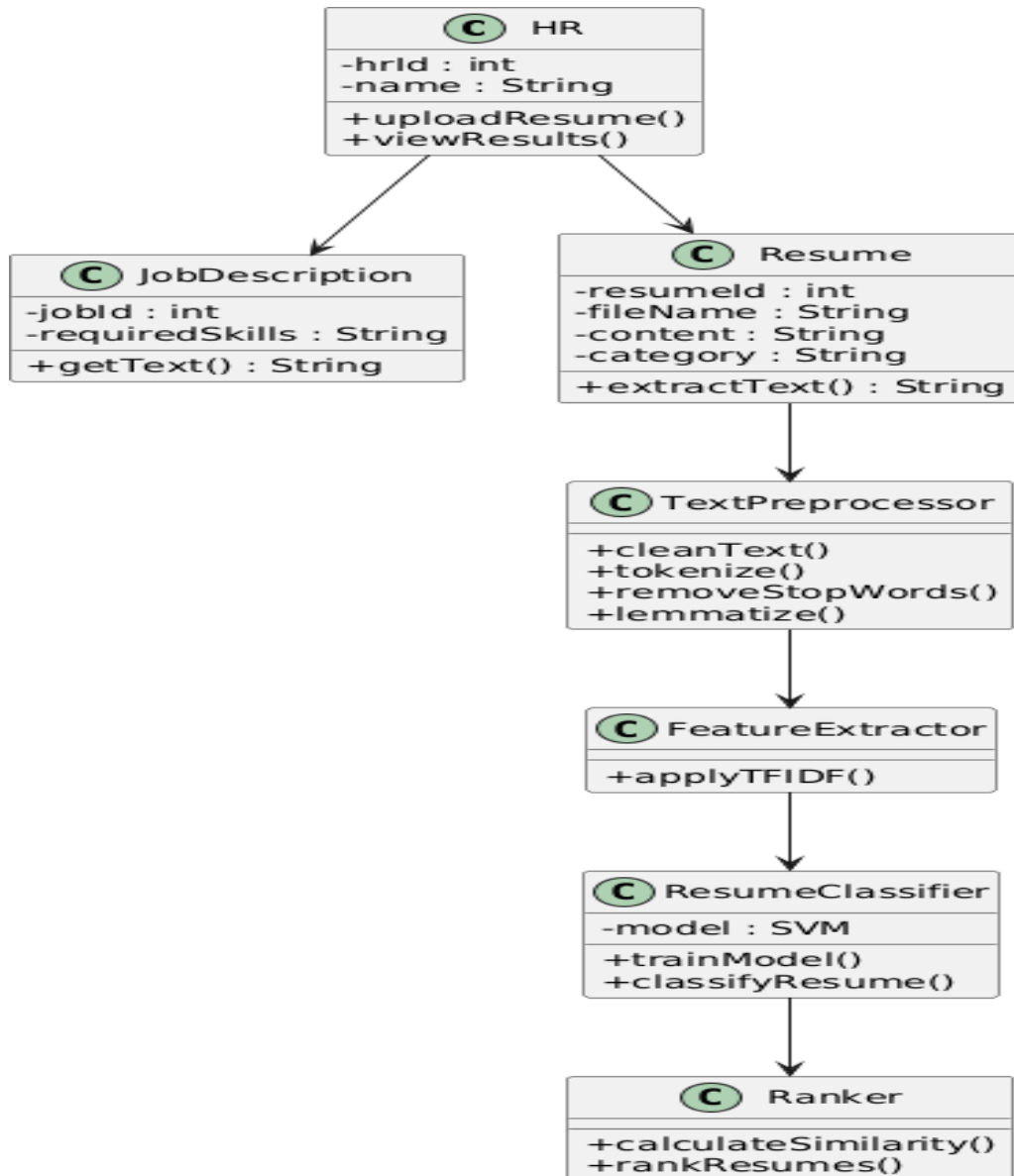


Fig: Class diagram

SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

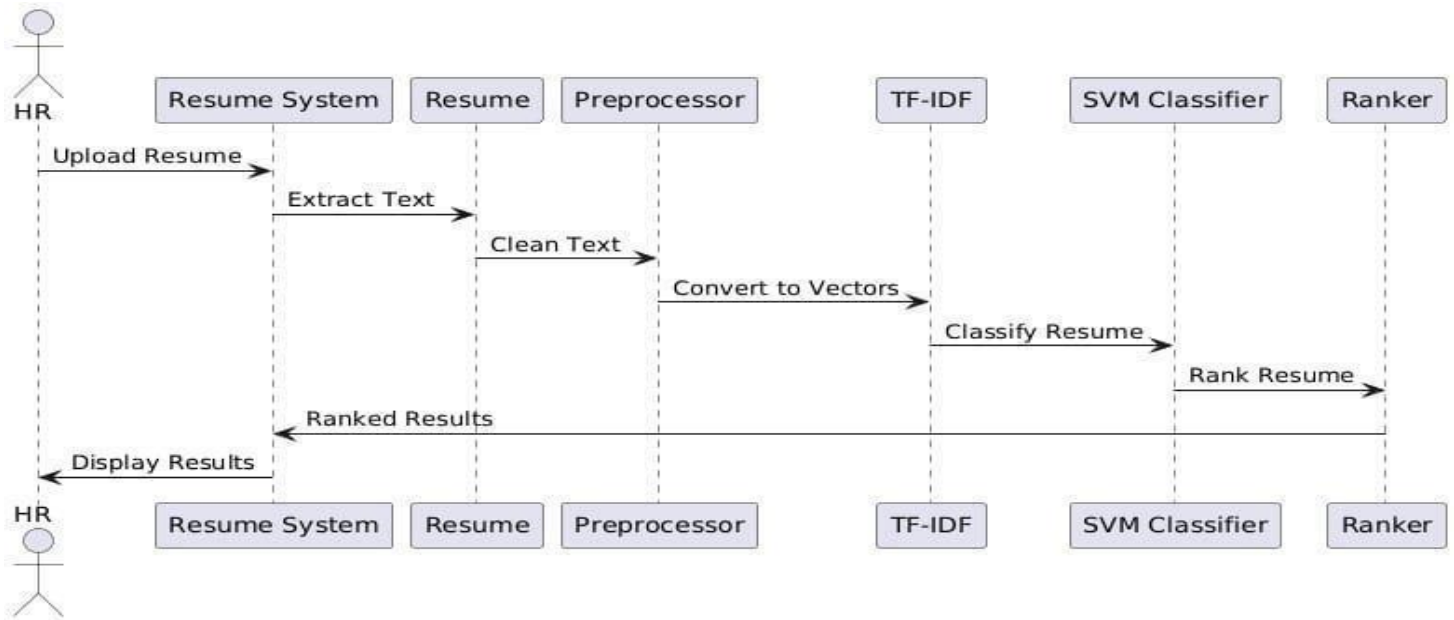


Fig: Sequence Diagram

Component Diagram :

A component diagram in UML (Unified Modeling Language) visually represents the static implementation view of a software system, focusing on the physical components and their dependencies. It shows how the system is built from distinct, deployable units, called components, and how they interact through interfaces. These diagrams are crucial for understanding the system's structure, identifying potential problems, and guiding development.

Resume Screening System using NLP - Component Diagram

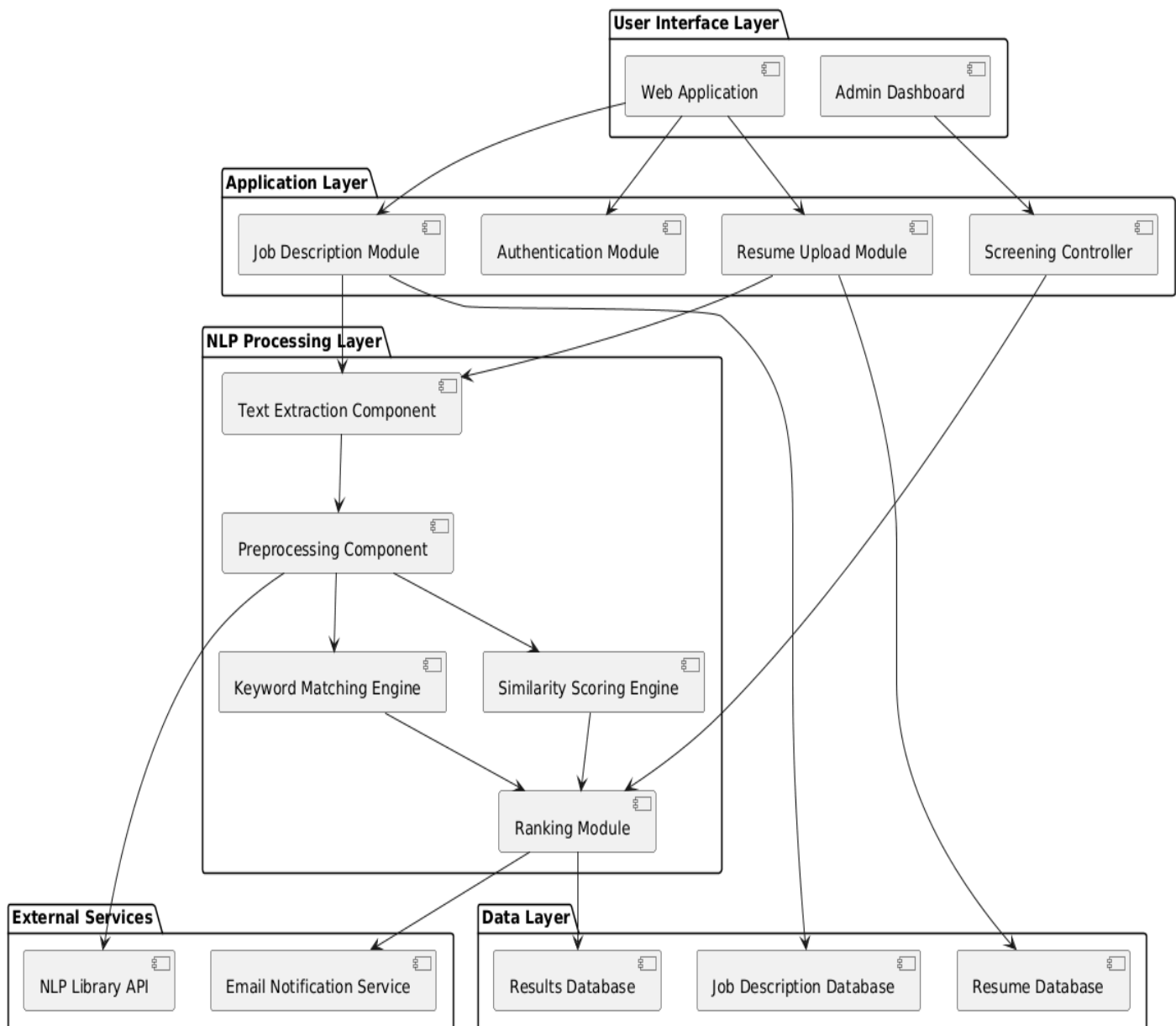


Fig: Component Diagram

Object Diagram

An object diagram in UML is a structural diagram that illustrates specific instances of classes and their relationships at a particular point in time. It's essentially a snapshot of a system's state, showing how objects (instances of classes) interact with each other. Unlike class diagrams which focus on the abstract structure of a system, object diagrams represent concrete instances of that structure.

Resume Screening System using NLP - Object Diagram (Domain: Data Science)

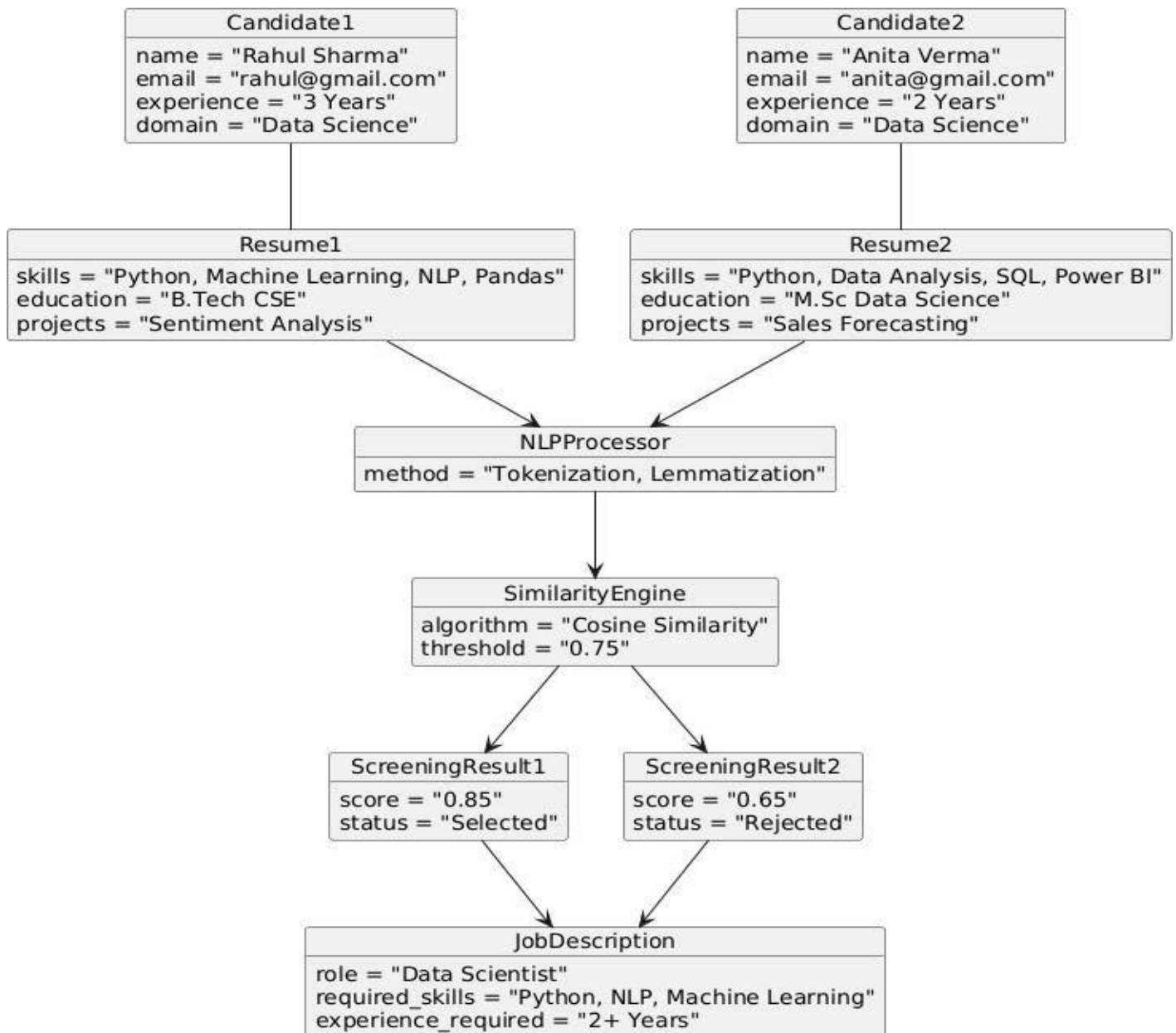


Fig: Object Diagram

CHAPTER 7

TECHNOLOGY DESCRIPTION AND IMPLEMENTATION

7. TECHNOLOGY DESCRIPTION AND IMPLEMENTATION

What is Python:

Below are some facts about Python.

1. Python is currently the most widely used multi-purpose, high-level programming language.
2. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.
3. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
4. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

Advantages of Python:

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be **extended to other languages**. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add **scripting capabilities** to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers **more productive** than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. Simple and Easy

When working with Java, you may have to create a class to print '**Hello World**'. But in Python, just a print statement will do. It is also quite **easy to learn, understand, and code**. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

6. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and **indentation is mandatory**. This further aids the readability of the code.

7. Object-Oriented

This language supports both the **procedural and object-oriented** programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the **encapsulation of data** and functions into one.

8. Free and Open-Source

Like we said earlier, Python is **freely available**. But not only can you **download Python** for free, but you can also download its source code, make changes to it, and even distribute it. It comes with an extensive collection of libraries to help you with your tasks.

9. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to **code only once**, and you can run it anywhere. This is called **Write Once Run Anywhere (WORA)**. However, you need to be careful enough not to include any system-dependent features.

10. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, **debugging is easier** than in compiled languages.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 GitHub annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python Everyone is for

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and **machinelearning**, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in **slow execution**. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbonnelle**.

The reason it is not so famous despite the existence of Bryton is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically-typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC(Java database Connectivity)** and **ODBC(Open Database Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary. This was all about the Advantages and Disadvantages of Python Programming Language.

HISTORY OF PYTHON:

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde&Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde end Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

INSTALLATION OF PYTHON:

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace. The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

Windows

Install Python from python.org. Use the **Download Python** button that appears first on the page

to download the latest version.

Note: If you don't have admin access, an additional option for installing Python on Windows is to use the Microsoft Store. The Microsoft Store provides installs of supported Python versions. For additional information about using Python on Windows, see Using Python on Windows at Python.org

MacOS

The system install of Python on macOS is not supported. Instead, a package management system like Homebrew is recommended. To install Python using Homebrew on macOS use `brew install python3` at the Terminal prompt.

Note: On macOS, make sure the location of your VS Code installation is included in your PATH environment variable. See these setup instructions for more information.

Linux

The built-in Python 3 installation on Linux works well, but to install other Python packages you must install pip with [get-pip.py](https://pypi.org/project/get-pip/).

Step 1: Go to the official site to download and install python using Google Chrome.

OR Click on the following link: <https://www.python.org>



Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Release version	Release date		Click for more
Python 3.10.14	March 19, 2024	 Download	Release Notes
Python 3.9.19	March 19, 2024	 Download	Release Notes
Python 3.8.19	March 19, 2024	 Download	Release Notes
Python 3.11.8	Feb. 6, 2024	 Download	Release Notes
Python 3.12.2	Feb. 6, 2024	 Download	Release Notes
Python 3.12.1	Dec. 8, 2023	 Download	Release Notes
Python 3.11.7	Dec. 4, 2023	 Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

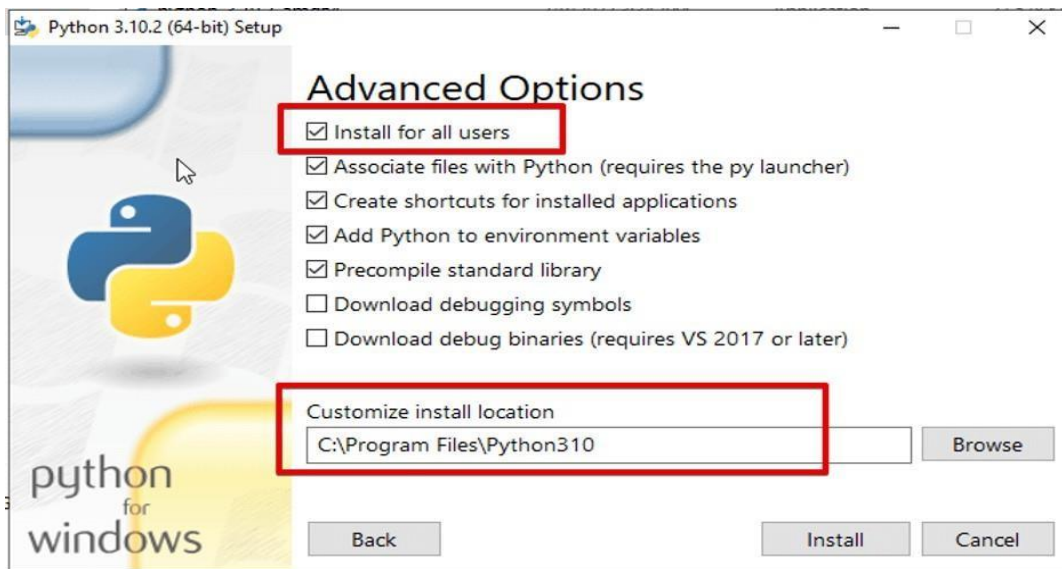
- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer
- To download Windows 64-bit python, you can select anyone from the three options: Windows x86 64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer. version and then install the new one.

Installation of Python

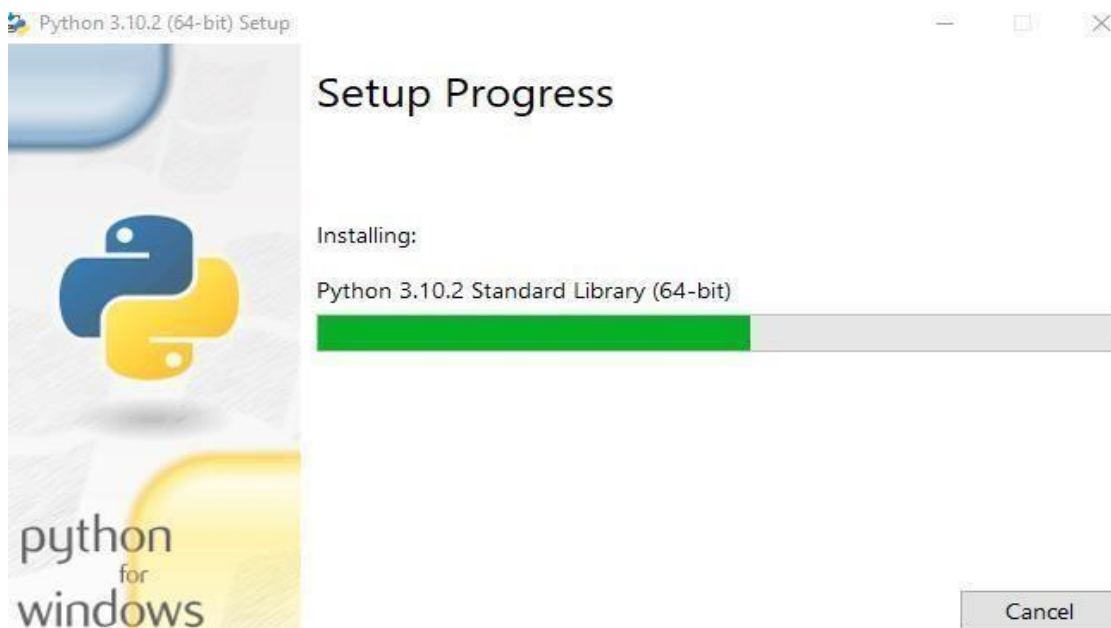
Step 1: Go to Download and Open the downloaded python version to carryout the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.10.2 to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.



With these above three steps on python installation, you have successfully and correctly installedpython
Now is the time to verify the installation.

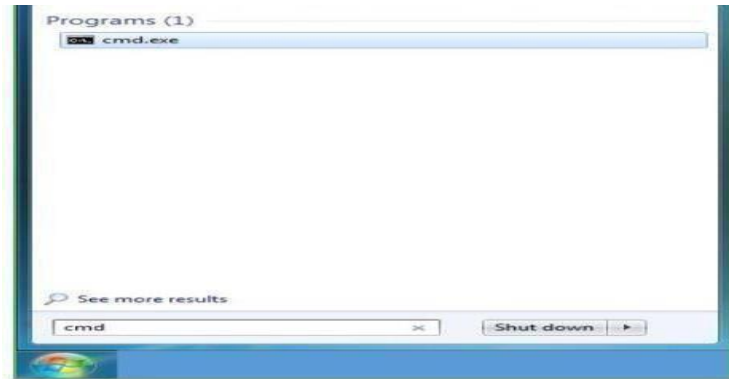
Note: The installation process might take a couple of minutes

Verify the Python

Installation Step 1: Click

on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python -V and press Enter.

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.20348.2582]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator>python --version
Python 3.10.2

C:\Users\Administrator>_
```

Step 5: You will get the answer as 3.10.2

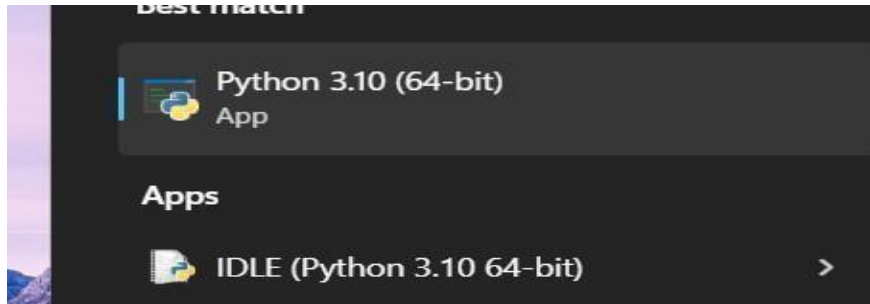
Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier

version and then install the new one.

Check how the Python IDLE works

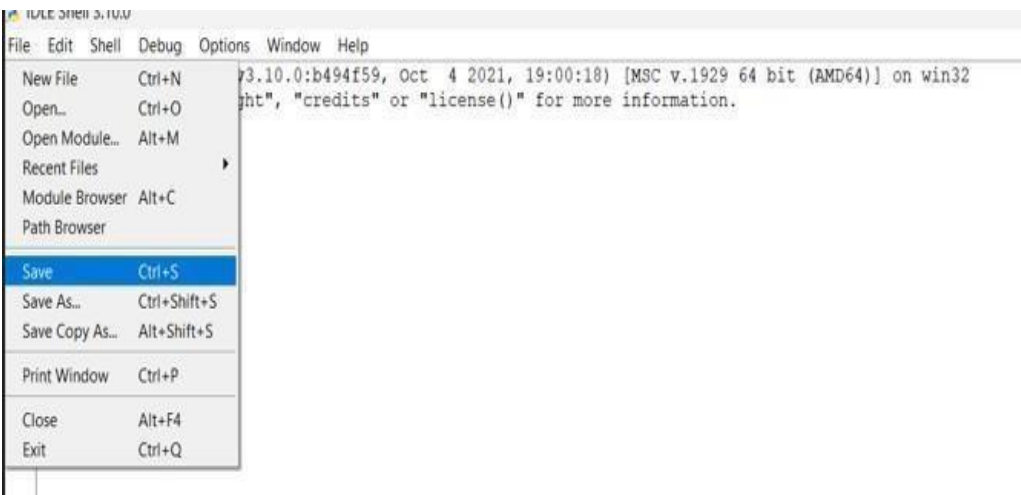
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



Step 3: Click on IDLE (Python 3.10 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World

Step 6: Now for e.g. **enter print**

What is Node.js?

Node.js is an open-source, server-side runtime environment that allows developers to run JavaScript outside the web browser. It is built on Google Chrome's V8 JavaScript engine and is mainly used to develop fast and scalable backend applications. Node.js follows an event-driven and non-blocking I/O model, which makes it efficient for handling multiple user requests at the same time.

It is commonly used for building web servers, APIs, real-time applications like chat systems, and

data-driven applications. With the help of NPM (Node Package Manager), developers can easily install and use thousands of reusable packages to speed up development.

A Glimpse into Node.js Philosophy

Node.js philosophy is centered around simplicity, efficiency, and scalability. It follows the principle of “**JavaScript everywhere,**” meaning the same language can be used for both frontend and backend development. One of the core ideas behind Node.js is its **event-driven and non-blocking I/O model**, which allows the system to handle multiple requests simultaneously without waiting for one task to finish before starting another. This makes it highly efficient and suitable for real-time and data-intensive applications.

Another important philosophy of Node.js is **lightweight and modular design**. Instead of building everything into the core system, Node.js relies on small, reusable modules that can be installed using NPM (Node Package Manager). This encourages flexibility and faster development.

Node.js also promotes **single-threaded but highly scalable architecture**, where a single thread manages multiple operations using an event loop. This reduces resource usage while maintaining high performance. Overall, Node.js philosophy focuses on building fast, scalable, and modern web applications using simple and efficient design principles.

Key Features of Node.js

Node.js functionality can be summarized as follows:

- **Asynchronous and Non-Blocking I/O:** Node.js processes multiple requests at the same time without waiting for one task to finish, which improves performance and speed.
- **Event-Driven Architecture:** It uses an event loop mechanism to handle requests efficiently, making it suitable for real-time applications like chat apps and online games.
- **Fast Execution:** Built on Google Chrome’s V8 JavaScript engine, Node.js compiles JavaScript into machine code, making it very fast.
- **Single-Threaded but Scalable:** Even though it uses a single thread, it can handle thousands of concurrent connections using event-driven programming.
- **Cross-Platform:** Node.js applications can run on Windows, Linux, macOS without changes.
- **Open Source:** Node.js is free to use and has strong community support worldwide.

- **Large Ecosystem (NPM):** Node Package Manager (NPM) provides thousands of reusable libraries and modules that speed up development.
- **Real-Time Data Handling:** Node.js is highly suitable for real-time applications like chat applications, live notifications, and online collaboration tools.
- **JSON Support:** It works smoothly with JSON (JavaScript Object Notation), which makes data exchange between client and server simple and fast.
- **Highly Extensible:** Developers can easily add new functionalities using external modules and libraries through NPM.
- **Microservices Friendly:** Node.js is well-suited for building microservices architecture, where applications are divided into small, independent services.

Need for Node.js

Simplicity and Flexibility:

- **Efficient:** Today's applications need real-time interaction. Node.js is needed to efficiently manage continuous data exchange between client and server.
- **Accessibility:** Developers prefer using a single programming language for both frontend and backend to reduce complexity. Node.js makes this possible using JavaScript everywhere.
- **Scalability:** Modern web applications require handling thousands of users at the same time. Traditional server technologies may become slow, so Node.js is needed for better scalability.
- **Reusability:** Businesses require faster development and deployment. Node.js helps reduce development time with reusable packages and simple architecture. applications.

Challenges in Node.js

1. **Error Handling Issues:** Managing errors in asynchronous code can be complex if not properly structured.
2. **Not Suitable for Heavy Computation:** Node.js is not ideal for CPU-heavy applications such as complex scientific calculations or large data processing tasks.
3. **API Instability:** Some Node.js packages may change frequently, which can cause compatibility problems in projects.
4. **Security Risks:** Using third-party modules without proper verification may introduce vulnerabilities.

Application of Node.js

Node.js versatility allows it to be used across various domains:

- **Web Applications** – Node.js is widely used to build fast and scalable web applications, especially backend servers and RESTful APIs.
- **Real-Time Applications** – It is ideal for chat applications, live notifications, online gaming, and collaboration tools because it supports real-time data exchange.
- **Streaming Applications** – Node.js is used for video and audio streaming platforms since it can handle data in chunks efficiently.
- **API Development** – It is commonly used to create lightweight and high-performance APIs for web and mobile applications.
- **Single Page Applications (SPAs)** – Node.js supports backend services for modern frontend frameworks like React and Angular.
- **Microservices Architecture** – It is suitable for building small, independent services that work together in large applications.
- **E-commerce Platforms** – Many online shopping websites use Node.js for managing large user traffic and transactions.

Machine Learning libraries

1. Scikit-Learn

Purpose: General-purpose Machine Learning library for classical algorithms.

Key Features:

- Provides implementations of regression, classification, clustering, dimensionality reduction
- Offers utilities for model selection, preprocessing, and validation.
- Includes APIs for splitting datasets, scaling features, and hyperparameter tuning.

Common Functions in Use:

- `Train_test_split`: Splits datasets into training and testing subsets.
- `Standard Scaler`: Standardizes features by removing the mean and scaling to unit variance.
- Algorithms like Linear Regression, Random Forest Classifier, SVC(Support Vector Classifier).

TensorFlow and Keras

Purpose: Deep learning and Neural Networks.

Key Features:

- TensorFlow is a comprehensive library for large-scale Machine Learning and deep learning tasks.
- Keras, integrated within TensorFlow, provides a high-level API for building neural networks.
- Suitable for tasks like image recognition, natural language processing, and reinforcement learning.

Common Functions in Use:

- Sequential: Simplifies the creation of sequential neural networks.
- Layers like Dense, Conv2D, LSTM.
- Utilities like fit for training, evaluate for validation, and predict for inference.

2. PyTorch

Purpose: Deep Learning with dynamic computation graphs.

Key Features:

- Offers flexibility with dynamic graph computation, enabling easier debugging and customization.
- Suitable for research and production-level deployments.
- Commonly used for applications like image and sequence modeling.

Common Functions in Use:

- torch.: Provides neural network layers and utilities.
- Data Loader: Facilitates batch processing of datasets.
- optim: Provides optimization algorithms like SGD and Adam.

3. Pandas

Purpose: Data manipulation and analysis.

Key Features:

- Provides data structures like DataFrame and Series for working with structured datasets.
- Supports operations like data cleaning, manipulation, and aggregation.

Common Functions in Use:

- read_csv: Reads CSV files into a DataFrame.

- groupby, merge, pivot: Facilitates data analysis and preprocessing.
- head, info, describe: Used for exploratory data analysis.

4. NumPy

Purpose: Numerical computing and array manipulation.

Key Features:

- Provides multi-dimensional arrays and high-performance mathematical operations.
- Often used as the foundation for other libraries like Pandas, Scikit-Learn, and TensorFlow.

Common Functions in Use:

- array, reshape, linspace: Manipulating numerical arrays.
- mean, std, dot: Statistical and matrix operations.

5. Matplotlib and Seaborn

Purpose: Data visualization.

Key Features:

- **Matplotlib:** Core library for plotting graphs and visualizing data.
- **Seaborn:** Built on top of Matplotlib, provides a high-level API for statistical data visualization.

Common Functions in Use:

- plot, scatter, bar: For basic visualizations (Matplotlib).
- heatmap, pair plot, boxplot: For advanced visualizations (Seaborn).

6. NLTK and SpaCy

Purpose: Natural Language Processing (NLP).

Key Features:

- **NLTK:** Great for academic tasks like tokenization, stemming, lemmatization, and basic text processing.
- **SpaCy:** Industrial-grade NLP library for named entity recognition, part-of-speech tagging, and dependency parsing.

Common Functions in Use:

- NLTK's word_tokenize, sentiment_analysis.

- SpaCy'snlp, docents for entity recognition.

7. OpenCV

Purpose: Computer vision tasks.

Key Features:

- Facilitates image processing, object detection, and video analysis.
- Works seamlessly with deep learning libraries like TensorFlow or PyTorch for visual recognition tasks.

Common Functions in Use:

- cv2.imread, cv2.imshow: For reading and displaying images.
- cv2.Canny: For edge detection.
- cv2.HOGDescriptor: For feature extraction.

8. Joblib or Pickle

Purpose: Saving and loading Machine Learning models.

Key Features:

- Serializes Python objects like trained models.
- Ensures compatibility when integrating with Django or Flask for API deployment.

Common Functions in Use:

- joblib.dump: Saves the model.
- joblib.load: Loads the model.

Data Science

Introduction

Data Science is an interdisciplinary field that combines statistics, computer science, mathematics, and domain expertise to extract meaningful insights from structured and unstructured data. The goal of data science is to analyze large volumes of data, discover hidden patterns, generate insights, and support data-driven decision-making.

While machine learning focuses mainly on building predictive models, data science covers the entire lifecycle of data — from data collection and cleaning to visualization and deployment of insights.

In today's digital world, organizations generate massive amounts of data through websites, mobile applications, sensors, transactions, and social media. Data science helps convert this raw data into actionable knowledge.

For example:

- E-commerce platforms analyze customer purchase history to recommend products.
- Banks use data science to detect fraudulent transactions.
- Healthcare institutions analyze patient records to predict diseases.
- Streaming services suggest movies and shows based on viewing behavior.
- Data science is continuously evolving with advancements in artificial intelligence (AI), big data technologies, and cloud computing.

Machine Learning Methods

In machine learning, tasks are generally classified into broad categories. These categories are based on how learning is received or how feedback on the learning is given to the system developed. Two of the most widely adopted machine learning methods are **supervised learning** which trains algorithms based on example input and output data that is labeled by humans, and **unsupervised learning** which provides the algorithm with no labeled data in order to allow it to find structure within its input data. Let's explore these methods in more detail.

Supervised Learning

In supervised learning, the computer is provided with example inputs that are labeled with their desired outputs. The purpose of this method is for the algorithm to be able to "learn" by comparing its actual output with the "taught" outputs to find errors, and modify the model accordingly. Supervised learning therefore uses patterns to predict label values on additional unlabeled data. For example, with supervised learning, an algorithm may be fed data with images of sharks labeled as fish and images of oceans labeled as water. By being trained on this data, the supervised learning algorithm should be able to later identify unlabeled shark images as fish and unlabeled ocean images as water. A common use case of supervised learning is to use historical data to predict statistically likely future events. It may use historical stock market information to anticipate

upcoming fluctuations, or be employed to filter out spam emails. In supervised learning, tagged photos of dogs can be used as input data to classify untagged photos of dogs

Unsupervised Learning

In unsupervised learning, data is unlabeled, so the learning algorithm is left to find commonalities among its input data. As unlabeled data are more abundant than labeled data, machine learning methods that facilitate unsupervised learning are particularly valuable.

The goal of unsupervised learning may be as straightforward as discovering hidden patterns within a dataset, but it may also have a goal of feature learning, which allows the computational machine to automatically discover the representations that are needed to classify raw data.

Unsupervised learning is commonly used for transactional data. You may have a large dataset of customers and their purchases, but as a human you will likely not be able to make sense of what similar attributes can be drawn from customer profiles and their types of purchases. With this data fed into an unsupervised learning algorithm, it may be determined that women of a certain age range who buy unscented soaps are likely to be pregnant, and therefore a marketing campaign related to pregnancy and baby products can be targeted to this audience in order to increase their number of purchases.

Without being told a “correct” answer, unsupervised learning methods can look at complex data that is more expansive and seemingly unrelated in order to organize it in potentially meaningful ways. Unsupervised learning is often used for anomaly detection including for fraudulent credit card purchases, and recommender systems that recommend what products to buy next. In unsupervised learning, untagged photos of dogs can be used as input data for the algorithm to find likenesses and classify dog photos together.

Approaches

As a field, machine learning is closely related to computational statistics, so having a background knowledge in statistics is useful for understanding and leveraging machine learning algorithms. For those who may not have studied statistics, it can be helpful to first define correlation and regression, as they are commonly used techniques for investigating the relationship among quantitative.

CHAPTER 8

SOURCE CODE

8.SOURCE CODE

#Importing packages

1. Importing Packages

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.io as pio
import joblib

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
import warnings
warnings.simplefilter(action="ignore")
pio.templates.default = "plotly_dark"
```

2. Uploading DataSet (Generating Synthetic Resume Feature Data)

```
def generate_resume_data(n_samples=10000):
```

Simulates extracted features from parsed resumes:

- total_skills: Number of unique technical skills identified
- experience_years: Total years of professional experience
- keyword_density: Relevance of resume text to Job Description
- education_score: Numeric mapping of highest degree
- certification_count: Number of professional certifications

```
np.random.seed(42)
```

```
data = {
```

```
'total_skills': np.random.poisson(15, n_samples),
```

```
'experience_years': np.random.gamma(3, 2, n_samples).astype(int),
```

```
'avg_keyword_density': np.random.uniform(0.1, 0.9, n_samples),
```

```
'education_rank': np.random.choice([1, 2, 3, 4], n_samples, p=[0.1, 0.4, 0.4, 0.1]),
```

```
'certification_count': np.random.poisson(2, n_samples),
```

```
'parsing_confidence': np.random.uniform(0.7, 1.0, n_samples),
```

```

df = pd.DataFrame(data)
# LOGIC: Higher Skills + Experience + JD Keywords = Higher Ranking Score
score = 40
score += (df['total_skills'] / 2).clip(0, 25)
score += (df['experience_years'] * 2.5).clip(0, 25)
score += (df['avg_keyword_density'] * 15)
score += (df['education_rank'] * 2)
# Penalize if parsing confidence is low or skills are too few
score -= (df['total_skills'] < 5).astype(int) * 15
df['ranking_score'] = score.clip(0, 100)
return df
print("[INFO] Initializing Resume Intelligence Dataset...")
df = generate_resume_data(10000)
# Seeing Data Profile
df.info()
print("\nMissing Values Check:\n", df.isna().sum())
print("\nDuplicate Check:", df.duplicated().sum())

# 3. Performing EDA (Visualizations for Presentation)

# Distribution of Target Variable (ATS Ranking Score)
plt.figure(figsize=(10, 5))
sns.histplot(df['ranking_score'], bins=25, kde=True, color="#2ecc71")
plt.title('Distribution of Predicted ATS Ranking Scores')
plt.xlabel('Ranking Score (0-100)')
plt.ylabel('Frequency')
plt.show()

# Feature Distribution: Skill Count
sns.boxplot(x=df['total_skills'], color="orange")
plt.title('Extracted Skills Distribution Across Resumes')
plt.show()

# Quality Categorization for Visualization
df['candidate_tier'] = pd.cut(df['ranking_score'], bins=[0, 45, 75, 100], labels=['Standard', 'Competitive', 'premium'])
tier_counts = df['candidate_tier'].value_counts()

fig = px.pie(
    values=tier_counts.values,
    names=tier_counts.index,

```

```

template='plotly_dark',

title='Candidate Talent Pool Distribution',
hole=0.4
)
fig.update_traces(textinfo='label+percent')
fig.show()
# Relationship Analysis: Skills vs Ranking Score
plt.figure(figsize=(10, 5))
sns.scatterplot(data=df, x='total_skills', y='ranking_score', hue='candidate_tier', palette='viridis')
plt.title('Impact of Skill Richness on Ranking Score')
plt.show()

```

4. Handling Outliers (Data Cleaning)

```

numeric_columns = df.select_dtypes(include=['number']).columns
def outliers_handler(real_data, col_name):
    data = real_data.copy()
    q1 = np.percentile(data[col_name], 25)
    q3 = np.percentile(data[col_name], 75)
    iqr = q3 - q1
    upper_bound = q3 + (1.5 * iqr)
    lower_bound = q1 - (1.5 * iqr)
    outliers = (data[col_name] > upper_bound) | (data[col_name] < lower_bound)
    if len(data[outliers]) <= len(data) * 0.1:
        data = data[~outliers]
    else:
        data.loc[data[col_name] > upper_bound, col_name] = upper_bound
        data.loc[data[col_name] < lower_bound, col_name] = lower_bound
    return data
for i in numeric_columns:
    if i == 'ranking_score':
        break
df = outliers_handler(df, i)

```

5. Training And Testing (XGBoost Implementation)

```

# Encoding Quality Tiers
le = LabelEncoder()
df['tier_encoded'] = le.fit_transform(df['candidate_tier'])

```



```

x = df.drop(columns=['ranking_score', 'candidate_tier', 'tier_encoded'])
y = df['ranking_score']

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

# Correlation Analysis
plt.figure(figsize=(10, 8))
sns.heatmap(pd.concat([x_train, y_train], axis=1).corr(), annot=True, fmt='.2f', cmap='RdYlGn')
plt.title('Resume Feature Correlation Matrix')
plt.show()

# Scaling Features
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

# Model Training (XGBoost Regressor)
print("\n[TRAINING] Optimizing XGBoost Regressor for Resume Ranking...")
model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.03,
    max_depth=7,
    random_state=42
)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

# Evaluating Performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"\nModel Evaluation Metrics:")
print(f"Mean Squared Error: {mse:.2f}")
print(f"R2 Score (Variance Explained): {r2:.4f}")

# Save the trained model
joblib.dump(model, 'resume_ranking_model.pkl')
print("\n[SUCCESS] Production Model stored as resume_ranking_model.pkl")

```

CHAPTER 9

SYSTEM TESTING

9. SYSTEM TESTING

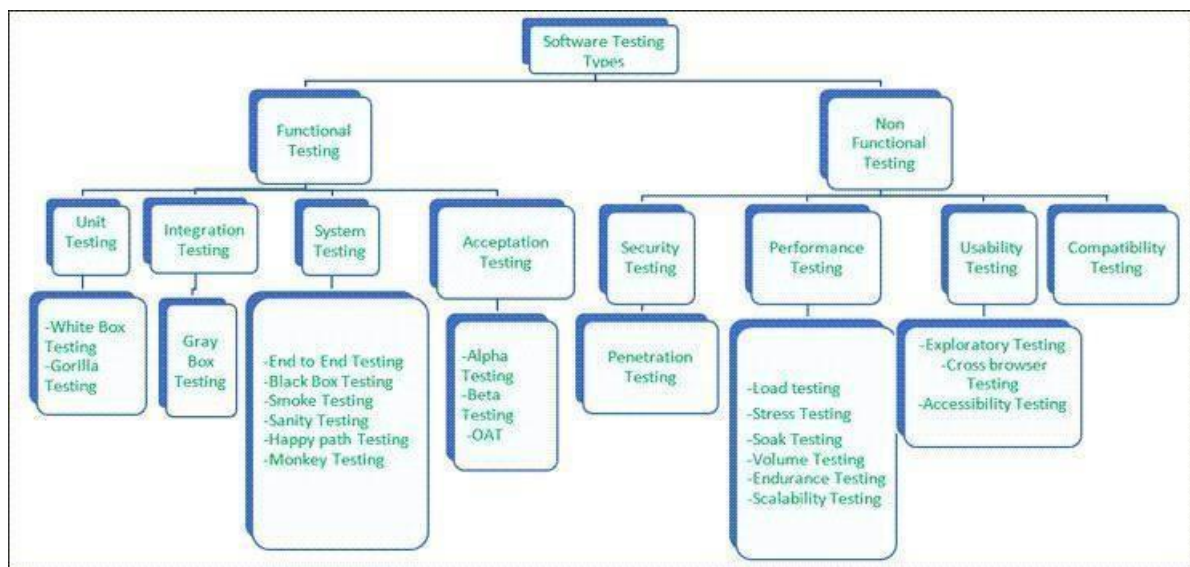
Types of Software Testing:

Different Testing Types with Details:

We, as testers, are aware of the various types of Software Testing like Functional Testing, Non-Functional Testing, Automation Testing, Agile Testing, and their sub-types, etc.

Each type of testing has its own features, advantages, and disadvantages as well.

Different Types of Software Testing



Functional Testing:

There are four main types of functional testing.

1) Unit Testing: Unit testing is a type of software testing which is done on an individual unit or component to test its corrections. Typically, Unit testing is done by the developer at the application development phase. Each unit in unit testing can be viewed as a method, function, procedure, or object. Developers often use test automation tools such as NUnit, Xunit, JUnit for the test execution.

2) White Box Testing: White box testing is a test technique in which the internal structure or code of an application is visible and accessible to the tester. In this technique, it is easy to find loopholes in the design of an application or fault in business logic. Statement coverage and decision coverage/branch coverage are examples of white box test techniques.

a) **Gorilla Testing:** Gorilla testing is a test technique in which the tester and/or developer test the module the application thoroughly in all aspects. Gorilla testing is done to check how robust your application is. **For example**, the tester is testing the pet insurance company's website, which provides the service of buying an insurance policy, tag for the pet, Lifetime membership.

3) Integration Testing: Integration testing is a type of software testing where two or more modules of an application are logically grouped together and tested as a whole. The focus of this type of testing is to find the defect on interface, communication, and data flow among modules. Top-down or Bottom-up approach is used while integrating modules into the whole system. This type of testing is done on integrating modules of a system or between systems. **For example**, a user is buying a flight ticket from any airline website.

Gray box testing: As the name suggests, gray box testing is a combination of white-box testing and blackbox testing. Testers have partial knowledge of the internal structure or code of an application.

4) System Testing: System testing is types of testing where tester evaluates the whole system against the specified requirements.

a) **End to End Testing:** It involves testing a complete application environment in a situation that mimics real-world use, such as interacting with a database, using network communications, or interacting with other hardware, applications, or systems if appropriate.

Black Box Testing: Blackbox testing is a software testing technique in which testing is performed without knowing the internal structure, design, or code of a system under test. Testers should focus only on the input and output of test objects. Detailed information about the advantages, disadvantages, and types of Black Box testing can be found here.

b) **Smoke Testing:** Smoke testing is performed to verify that basic and critical functionality of the system under test is working fine at a very high level. Whenever a new build is provided by the development team, then the Software Testing team validates the build and ensures that no major issue exists.

The testing team will ensure that the build is stable, and a detailed level of testing will be carried out further. **For example**, tester is testing pet insurance website. Buying an insurance policy, adding another pet, providing quotes are all basic and critical functionality of the application.

c) **Sanity Testing:** Sanity testing is performed on a system to verify that newly added functionality or bug fixes are working fine. Sanity testing is done on stable build. It is a subset of the regression test.

For example, a tester is testing a pet insurance website. There is a change in the discount for buying a policy for second pet. Then sanity testing is only performed on buying insurance policy module.

d) **Happy path Testing:** The objective of Happy Path Testing is to test an application successfully on a positive flow. It does not look for negative or error conditions.

e) **Monkey Testing:** Monkey Testing is carried out by a tester, assuming that if the monkey uses the application, then how random input and values will be entered by the Monkey without any knowledge or understanding of the application. The objective of Monkey Testing is to check if an application or system gets crashed by providing random input values/data.

5) Acceptance Testing: Acceptance testing is a type of testing where client/business/customer test the software with real time business scenarios. The client accepts the software only when all the feature and functionalities work as expected. This is also called User Acceptance Testing (UAT).

a) **Alpha Testing:** Alpha testing is a type of acceptance testing performed by the team in an organization to find as many defects as possible before releasing software to customers. **For example**, the pet insurance website is under UAT.

b) **Beta Testing:** Beta Testing is a type of software testing which is carried out by the clients/customers. It is performed in the **Real Environment** before releasing the product to the market for the actual endusers. Beta Testing is carried out to ensure that there are no major failures in the software or product, and it satisfies the business requirements from an end-user perspective. Usually, this testing is typically done by the end-users. This is the final testing done before releasing the application for commercial purposes. Usually, the Beta version of the software or product released is limited to a certain number of users in a specific area.

c) **Operational acceptance testing (OAT):**

Operational acceptance testing of the system is performed by operations or system administration staff in the production environment. The purpose of operational acceptance testing is to make sure

that the system administrators can keep the system working properly for the users in a real- time environment.

The focus of the OAT is on the following points:

- ☐ Testing of backup and restore.
- ☐ Installing, uninstalling, upgrading software. The
- ☐ recovery process in case of natural disaster.

Non-Functional Testing:

There are four main types of functional testing.

1) Security Testing: It is a type of testing performed by a special team. Any hacking method can penetrate the system. Security Testing is done to check how the software, application, or website is secure from internal and/or external threats. This testing includes how much software is secure from malicious programs, viruses and how secure & strong the authorization and authentication processes

a) **Penetration Testing:** Penetration Testing or Pen testing is the type of security testing performed as an authorized cyberattack on the system to find out the weak points of the system in terms of security. Pen testing is performed by outside contractors, generally known as ethical hackers. That is why it is also known as ethical hacking. Contractors perform different operations like SQL injection, URL manipulation,

Notes: Do not perform the Pen testing on your laptop/computer. Always take written permission to do pen tests.

2) Performance Testing: Performance testing is testing of an application's stability and response time by applying load. The word stability means the ability of the application to withstand in the presence of load. Response time is how quickly an application is available to users. Performance testing is done with the help of tools. Loader.IO, JMeter, LoadRunner, etc. are good tools available in the market.

a) **Load testing:** Load testing is testing of an application's stability and response time by applying load, which is equal to or less than the designed number of users for an application. **For example,** your application handles 100 users at a time with a response time of 3 seconds, then load testing can be done by applying a load of the maximum of 100 or less than 100 users.

b) **Stress Testing:** Stress testing is testing an application's stability and response time by applying load, which is more than the designed number of users for an application.

For example, your application handles 1000 users at a time with a response time of 4 seconds, then stress testing can be done by applying a load of more than 1000 users.

c) Scalability Testing: Scalability testing is testing an application's stability and response time by applying load, which is more than the designed number of users for an application.**For example**, your application handles 1000 users at a time with a response time of 2 seconds, then scalability testing can be done by applying a load of more than 1000 users and gradually increasing the number of users to find out where exactly my application is crashing.

Let's say my application is giving response time as follows:

- ☐ 1000 users -2 sec
- ☐ 1400 users -2 sec
- ☐ 4000 users -3 sec
- ☐ 5000 users -45 sec
- ☐ 5150 users- crash – This is the point that needs to identify in scalability testing

d) Volume testing (flood testing):Volume testing is testing an application's stability and response time by transferring a large volume of data to the database. Basically, it tests the capacity of the database to handle the data.

e) Endurance Testing (Soak Testing):Endurance testing is testing an application's stability and response time by applying load continuously for a longer period to verify that the application is working fine.

Usability Testing: Usability testing is testing an application from the user's perspective to check the look and feel and user-friendliness.**For example**, there is a mobile app for stock trading, and a tester is performing usability testing. Testers can check the scenario like if the mobile app is easy to operate with one hand or not, scroll bar should be vertical, background colour of the app should be black and price of and stock is displayed in red or green colour.The main idea of usability testing of this kind of app is that as soon as the user opens the app, the user should get a glance at the market.

c) Exploratory testing: Exploratory Testing is informal testing performed by the testing team. The objective of this testing is to explore the application and look for defects that exist in the application. Testers use the knowledge of the business domain to test the application. Test charters are used to guide the exploratory testing.

d) Cross browser testing: Cross browser testing is testing an application on different browsers, operating systems, mobile devices to see look and feel and performance.

Why do we need cross-browser testing? The answer is different users use different operating systems, different browsers, and different mobile devices. The goal of the company is to get a good user experience regardless of those devices. Browser stack provides all the versions of all the browsers and all mobile devices to test the application. For learning purposes, it is good to take the free trial given by browser stack for a few days.

e) Accessibility Testing: The aim of Accessibility Testing is to determine whether the software or application is accessible for disabled people or not.

Here, disability means deafness, colour blindness, mentally disabled, blind, old age, and other disabled groups. Various checks are performed, such as font size for visually disabled, colour and contrast for colour blindness, etc.

3) Compatibility testing: This is a testing type in which it validates how software behaves and runs in a different environment, web servers, hardware, and network environment

TEST CASES

9.1 TEST CASES

SNO	Testcase Title	Pre-requisites	Action	Expected Result	Test Result (pass/fail)
Testcase1	Resume Upload	User logged in(Candidate / Recruiter/Admin in)	Upload resume in PDF/DOC format	Resume successfully uploaded and stored in database	Pass
Testcase2	Resume parsing	Resume Uploaded	System extracts text from resume using NLP parser	Resume text extracted correctly without data loss	Pass
Testcase3	Text Preprocessing	Extracted resume text available	Apply tokenization, stop word removal, lemmatization	Clean and structured Text generated for analysis	Pass
Testcase4	Skill Extraction	Preprocessed text available	Run NLP model to identify technical skills	Skills like Python, ML, SQL, Data Analytics is extracted correctly	Pass
Testcase5	Job description & Resume Matching	Resume and Job Description uploaded	Calculate similarity score using TF-IDF/Cosine Similarity	Match percentage displayed correctly(e.g.,75%)It	Pass

CHAPTER 10

INPUT AND OUTPUT DESIGN

10. INPUT AND OUTPUT DESIGN

Inputs:

Importing the all required packages like numpy , pandas, matplotlib, scikit – learn and required machine learning algorithms packages.

Outputs:

- Pre-processing the imported customer data for handling missing values and encoding categorical variables.
- Displaying cleaned and structured customer data.

INPUT DESIGN

Input design is a part of the overall system design. The main objective during the input design is as given below:

- To produce a cost-effective method of input.

OUTPUT DESIGN

Outputs from the system primarily serve to communicate the results of the analysis to stakeholders. They also provide a permanent record of predictions and insights.

Sample Input–Output Mapping

Input Type	Input Description	Output	System Result
Resume File	PDF/DOC resume uploaded	Extracted data	Resume parsed successfully
Resume Content	Skills, experience, education	Prediction + Confidence Score	Match score generated (e.g., 85%)
Invalid File	Corrupted/unsupported file	Error Message	"Invalid file format"
Admin CSV Upload	Dataset for training	Model Training	Retraining successful
Wrong Login	Incorrect credentials	Access Denied	"Invalid credentials"

INPUT PORTAL

An input portal for an AI-powered resume screening system using NLP. It allows users to securely enter details for intelligent analysis and candidate evaluation.

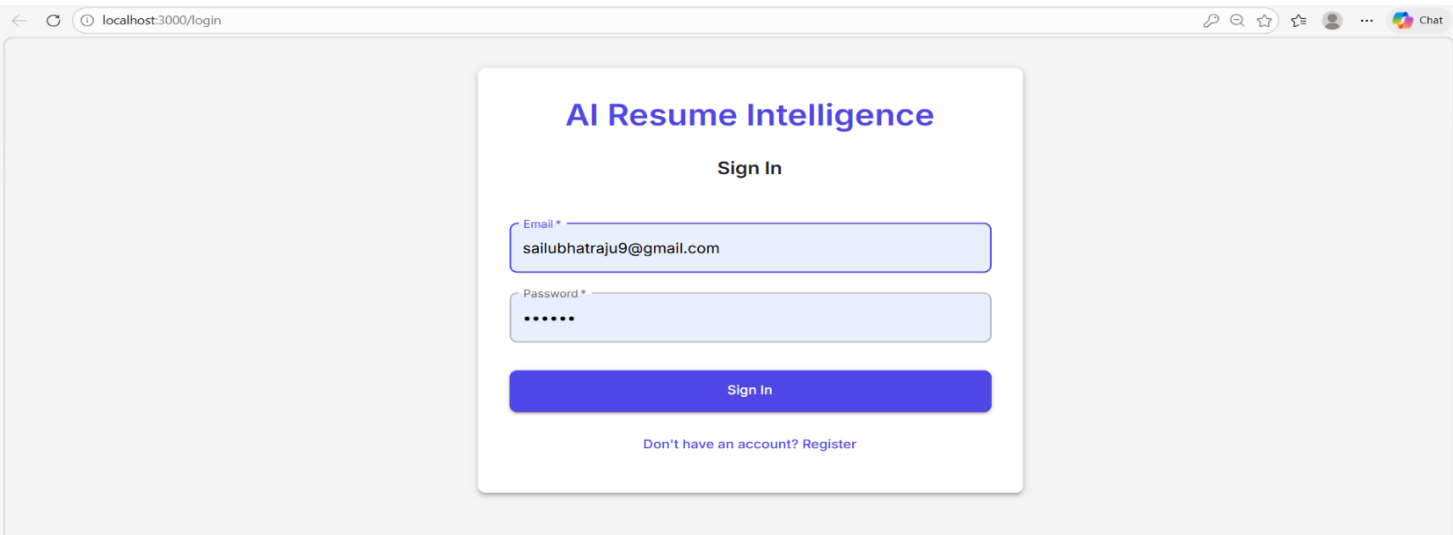


Fig: Input For Candidate portal

OUTPUT PORTAL

An output portal displaying system analytics of the AI-powered resume screening system. It provides insights on model performance, bias monitoring, processing statistics, and recruitment outcomes.

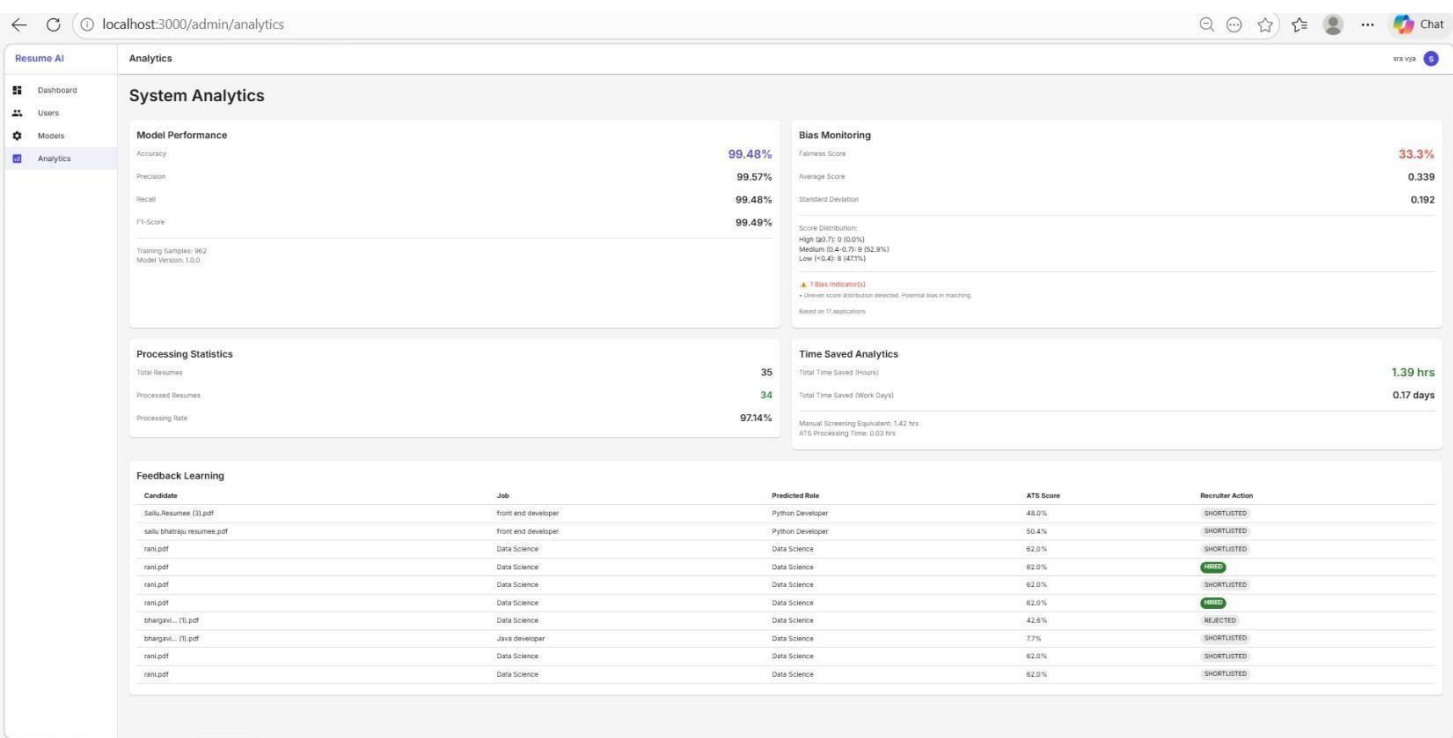


Fig: Output for admin portal

CHAPTER 11

RESULTS

11. OUTPUT SCREENSHOTS

DATASET:

```
dataset > ResumeDataSet.csv
1469
1470 Industry: Cross Industry
1471 Service Area: Cross Industry - Products
1472 Project Name: Handwriting recognition
1473 Consultant: 3 months.
1474 The project involved taking handwritten images and converting them to digital text images by
1475 Role: I was developing sentence correction functionality.
1476 Key Responsibilities:
1477   - Gather data large enough to capture all English words
1478   - Train LSTM models on words.
1479 Technical Environment: Python.
1480
1481 Industry: Finance
1482 Service Area: Financial Services - BI development Project Name: SWIFT
1483 Consultant: 8 months.
1484 The project was to develop an analytics infrastructure on top of SAP S/4, it would user to vi
1485 financial reports to respective departments. Reporting also included forecasting expenses.
1486 Role: I was leading the offshore team.
```

To run this project enter these commands,

```
PS C:\Users\sailu\OneDrive\Documents\Sailu\CSE_9\frontend> npm start
leware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
Compiled successfully!

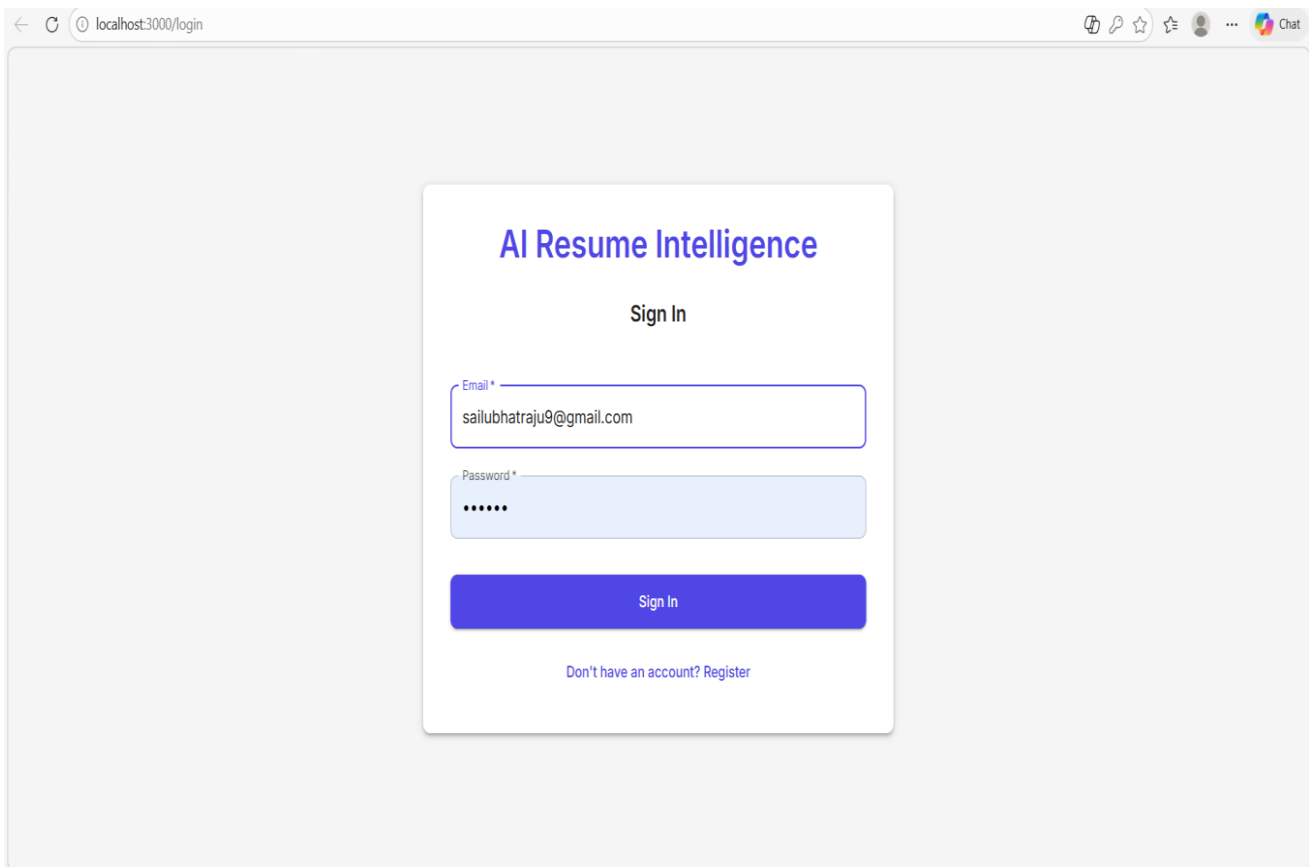
You can now view resume-intelligence-frontend in the browser.

Local:      http://localhost:3000
On Your Network: http://10.76.225.48:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
(node:5104) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assi
gn() instead.
```

INPUT

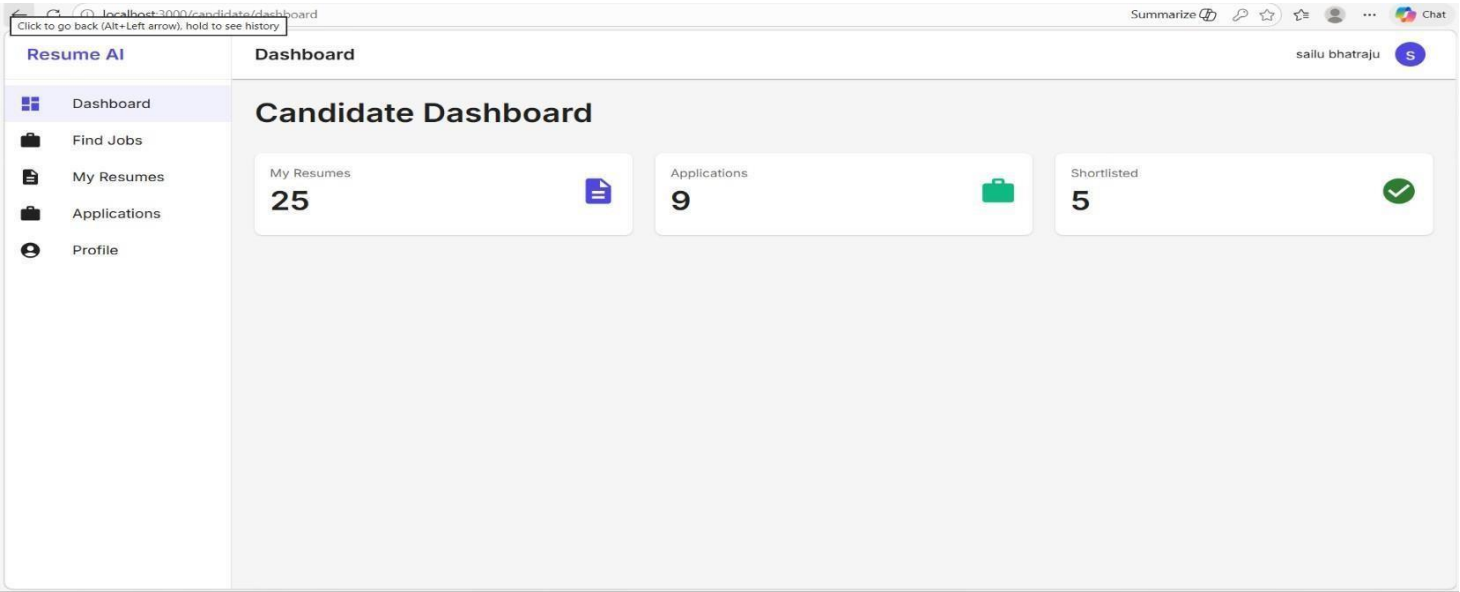


The screenshot shows a web browser window with the address bar displaying 'localhost:3000/login'. The page features a central white card with a blue border. At the top of the card, the text 'AI Resume Intelligence' is written in a bold, blue font. Below this, the heading 'Sign In' is centered. The card contains two input fields: the first is labeled 'Email *' and contains the text 'sailubhatraju9@gmail.com'; the second is labeled 'Password *' and contains seven dots. Below the password field is a solid blue button with the text 'Sign In' in white. At the bottom of the card, there is a link that reads 'Don't have an account? Register'.

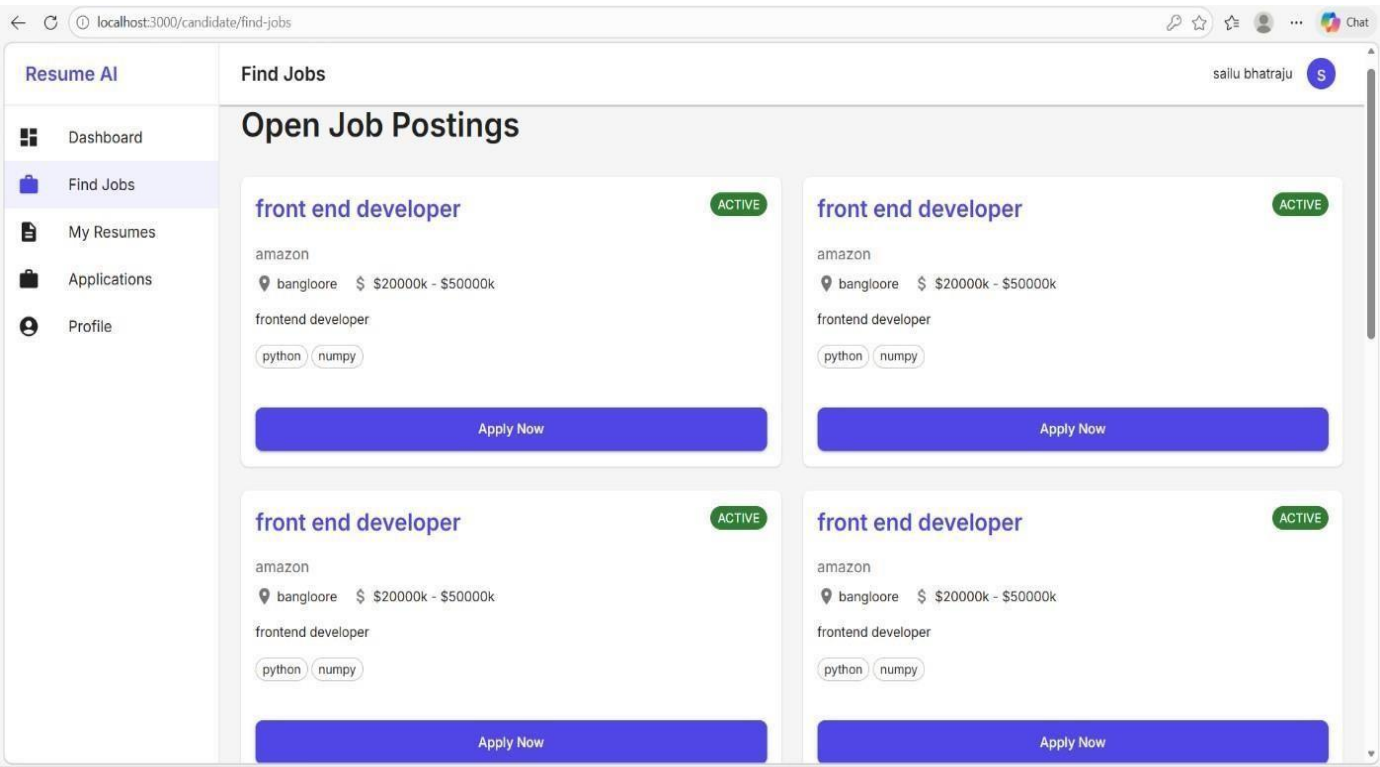
In this page, the user (such as a recruiter or HR manager) enters their **email and password** to sign in to the system. After logging in, the user can upload or analyze resumes. The system then uses **NLP techniques** to read and understand the text in resumes, such as skills, education, experience, and keywords.

Candidate Dashboard

candidate dashboard to track resumes, job applications, and shortlisted profiles efficiently.

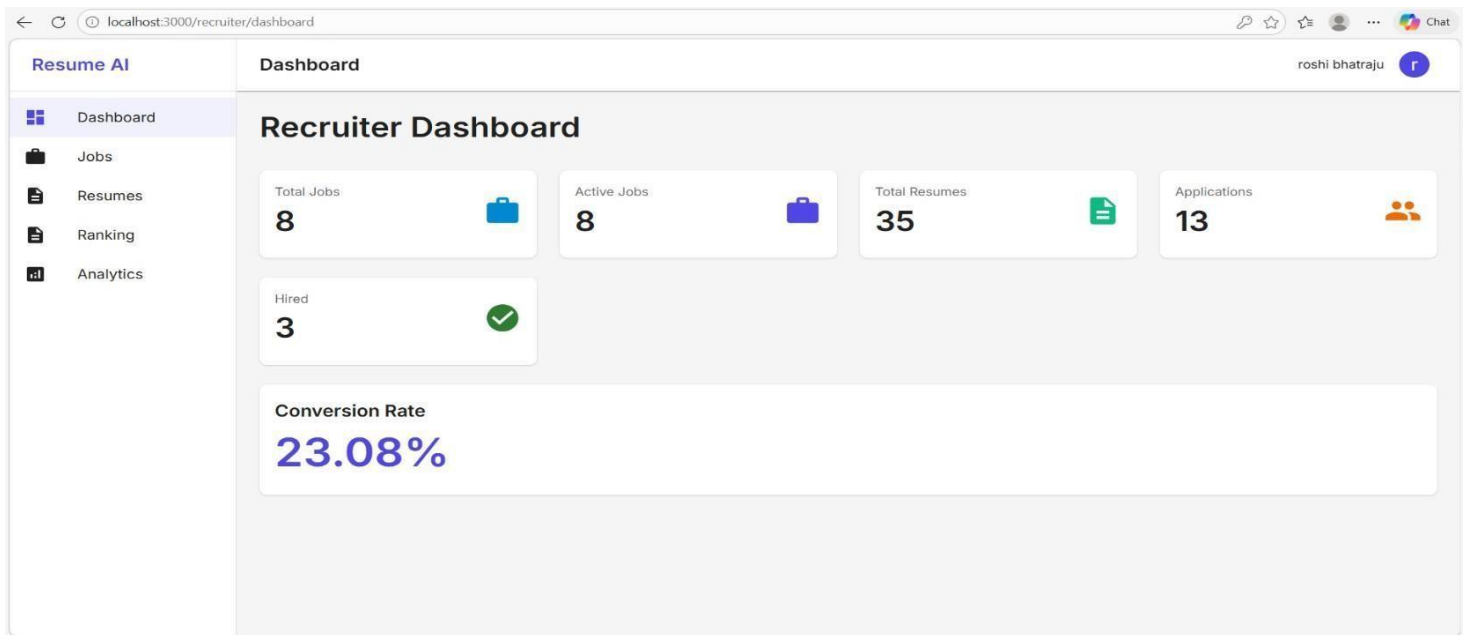


job postings, allowing candidates to explore roles and apply directly through the platform.



Recruiter Dashboard

recruiter dashboard to monitor jobs, resumes, applications, and hiring conversion rates.

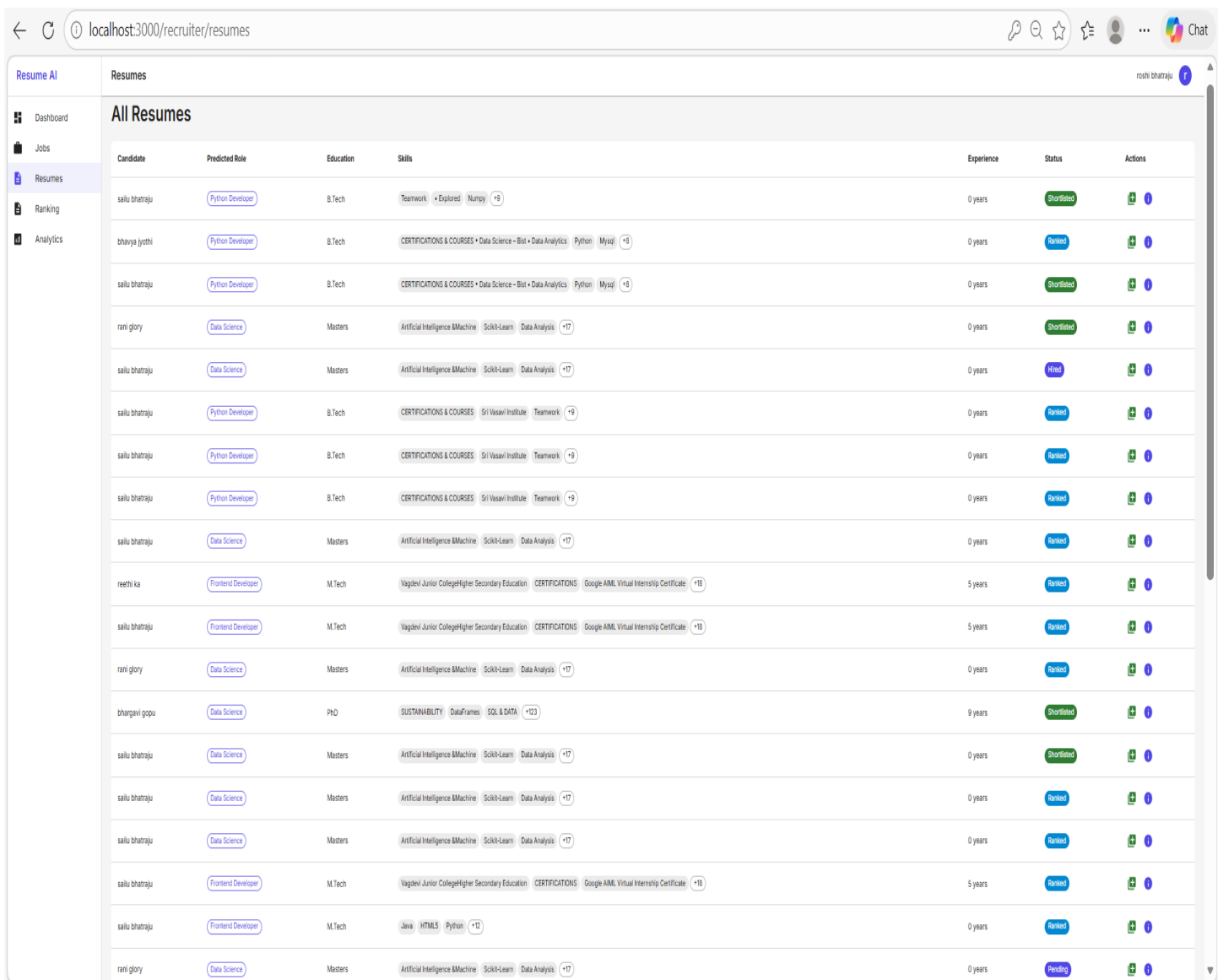


recruiters to manage and track job postings, required skills, experience levels, and application statuses efficiently.

The screenshot shows the 'Job Postings' table in the 'Jobs' section of the dashboard. The table lists various job openings with their details and actions.

Title	Skills	Experience	Status	Actions
front end developer	python numpy	1 years	active	
front end developer	python numpy	1 years	active	
front end developer	python numpy	1 years	active	
front end developer	python numpy	1 years	active	
front end developer	python numpy	1 years	active	
backend developer	python SQL Git	2 years	active	
Data Science	Pandas Numpy Tensorflow	0 years	active	
Java developer	ms office python HTML	0 years	active	

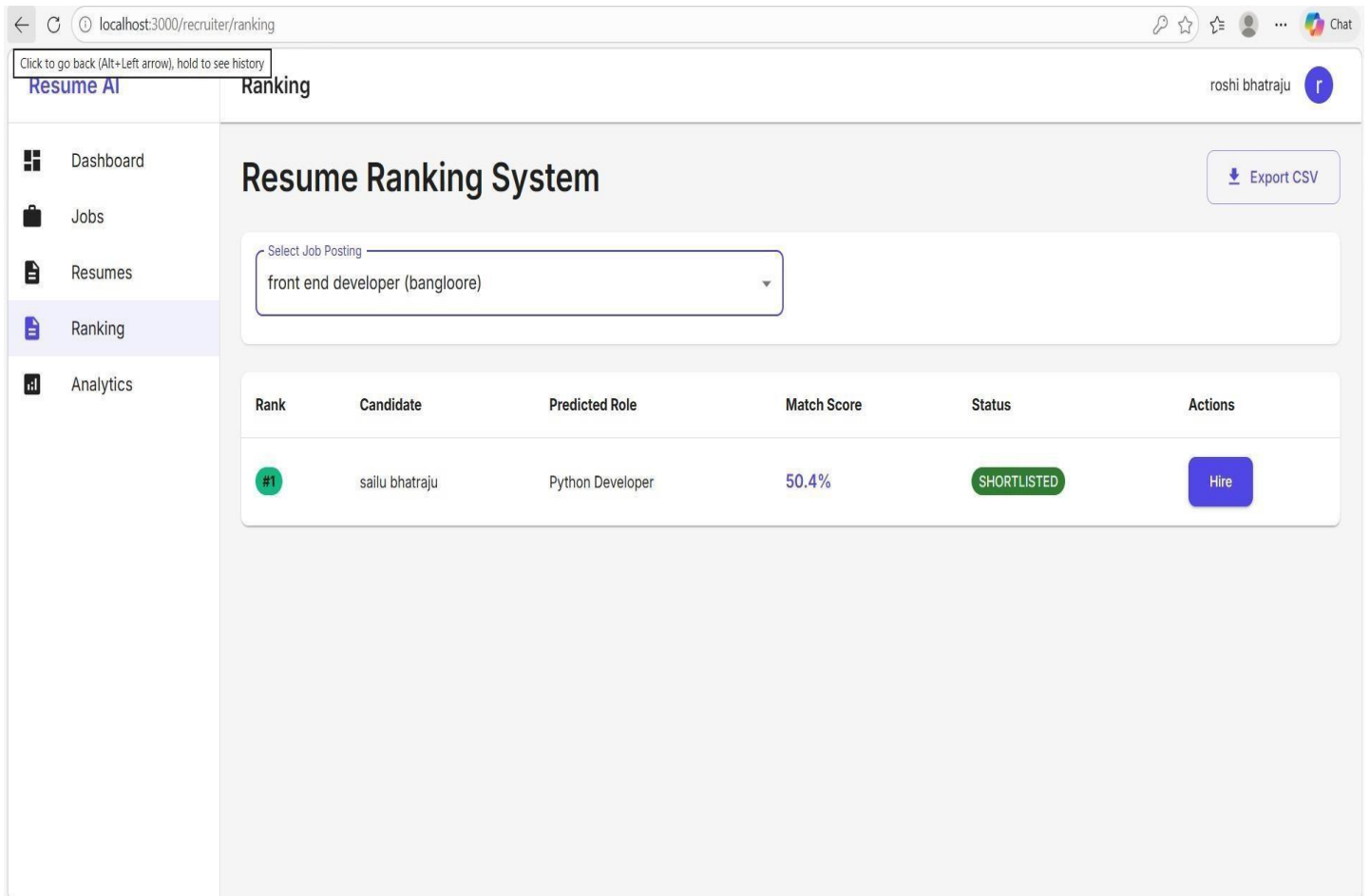
OUT PUT



All Resumes						
Candidate	Predicted Role	Education	Skills	Experience	Status	Actions
sailu bhatraju	Python Developer	B.Tech	Teamwork • Explored Numpy +9	0 years	Shortlisted	 
bhavya jyothis	Python Developer	B.Tech	CERTIFICATIONS & COURSES • Data Science - Bit • Data Analytics Python MySQL +8	0 years	Rejected	 
sailu bhatraju	Python Developer	B.Tech	CERTIFICATIONS & COURSES • Data Science - Bit • Data Analytics Python MySQL +8	0 years	Shortlisted	 
rani glory	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Shortlisted	 
sailu bhatraju	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Rejected	 
sailu bhatraju	Python Developer	B.Tech	CERTIFICATIONS & COURSES • Sri Vasavi Institute Teamwork +9	0 years	Rejected	 
sailu bhatraju	Python Developer	B.Tech	CERTIFICATIONS & COURSES • Sri Vasavi Institute Teamwork +9	0 years	Rejected	 
sailu bhatraju	Python Developer	B.Tech	CERTIFICATIONS & COURSES • Sri Vasavi Institute Teamwork +9	0 years	Rejected	 
sailu bhatraju	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Rejected	 
reethi ka	Frontend Developer	M.Tech	Vagdevi Junior College Higher Secondary Education CERTIFICATIONS Google AML Virtual Internship Certificate +18	5 years	Rejected	 
sailu bhatraju	Frontend Developer	M.Tech	Vagdevi Junior College Higher Secondary Education CERTIFICATIONS Google AML Virtual Internship Certificate +18	5 years	Rejected	 
rani glory	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Rejected	 
bhargavi gopu	Data Science	PhD	SUSTAINABILITY Dataframes SQL & DATA +13	9 years	Shortlisted	 
sailu bhatraju	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Shortlisted	 
sailu bhatraju	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Rejected	 
sailu bhatraju	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Rejected	 
sailu bhatraju	Frontend Developer	M.Tech	Vagdevi Junior College Higher Secondary Education CERTIFICATIONS Google AML Virtual Internship Certificate +18	5 years	Rejected	 
sailu bhatraju	Frontend Developer	M.Tech	Java HTML5 Python +12	0 years	Rejected	 
rani glory	Data Science	Masters	Artificial Intelligence & Machine Learning Data Analysis +17	0 years	Pending	 

This image shows the **All Resumes dashboard of the Resume Screening System using NLP**, where recruiters can view candidate details, predicted roles, skills, experience, and the screening status.

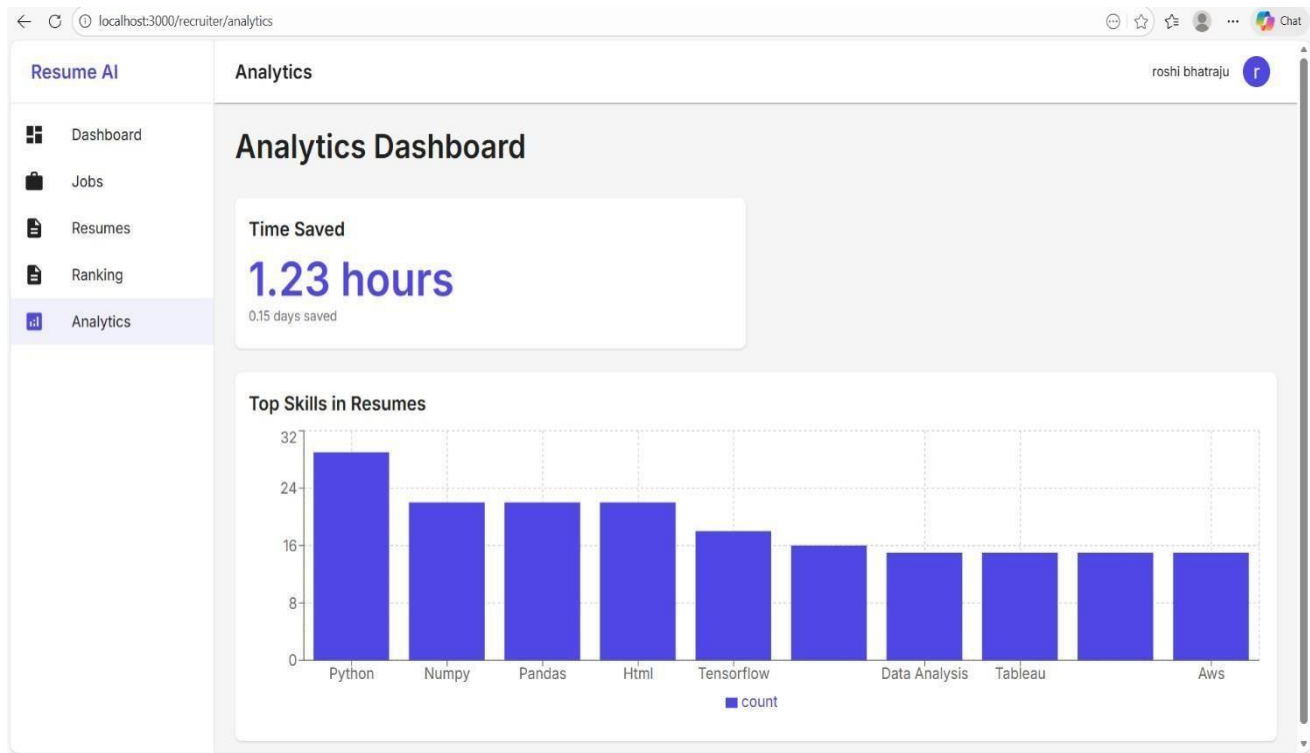
RESUME RANKING



This image shows the **Resume Ranking module of the Resume Screening System using NLP**.

The system analyzes uploaded resumes and compares them with the selected job posting. Using **Natural Language Processing techniques**, it evaluates candidate skills and experience to calculate a **match score**. Based on this score, candidates are **ranked and shortlisted**, helping recruiters quickly identify the most suitable applicants for the job.

Analytics Dashboard

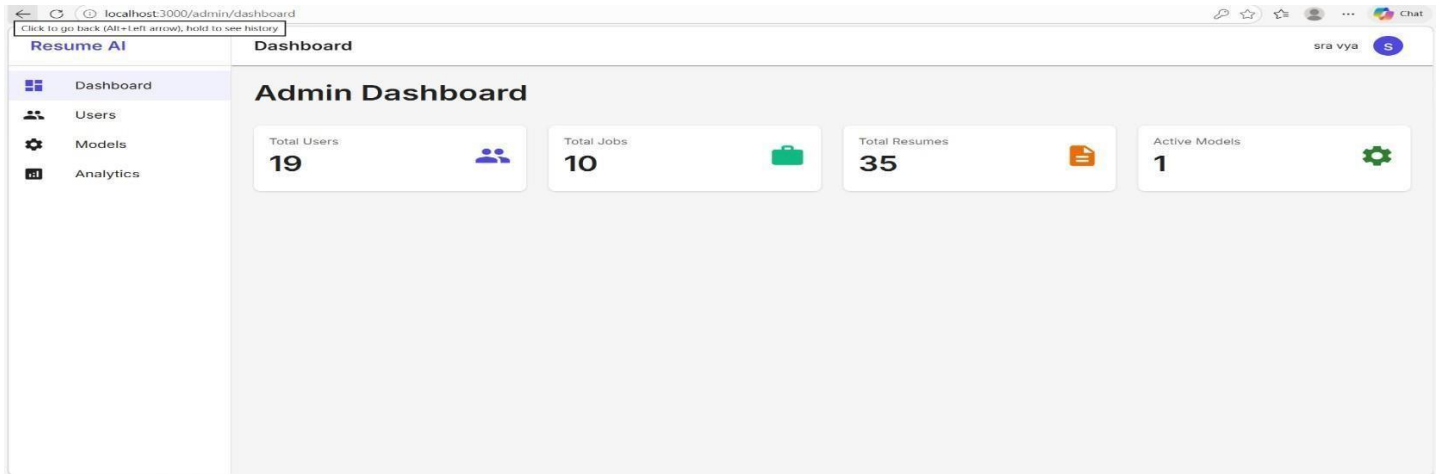


The image shows the Analytics Dashboard of the Resume AI system. It displays the **total time saved by automated resume screening using AI**. The chart shows **top skills found in resumes such as Python, Numpy, Pandas, HTML, and Machine Learning**.

This dashboard helps recruiters analyze candidate skills and recruitment efficiency.

Admin Dashboard

provides a centralized interface to monitor users, job postings, resumes, and machine learning models for automated recruitment management.



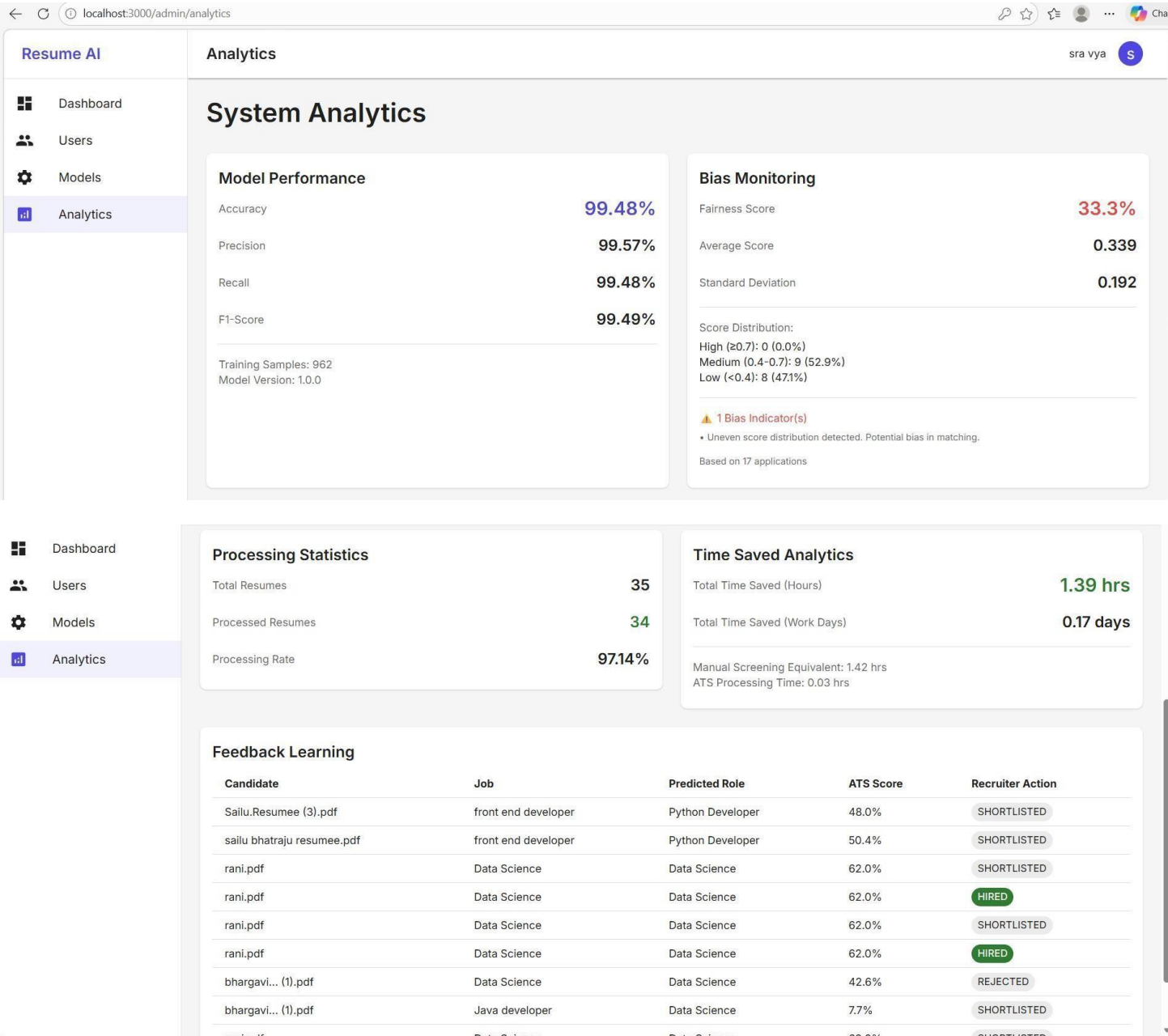
User Management

admin to manage candidate and recruiter accounts, including adding, updating, and monitoring user information.

The screenshot shows the Users Management page at localhost:3000/admin/users. The left sidebar is identical to the dashboard view, but the 'Users' menu item is selected. The main content area is titled 'Users Management' and displays a table of users. The table has columns for Name, Email, Role, Company, and Status. The user 'sra vya' is highlighted in blue.

Name	Email	Role	Company	Status
sasi boyina	sasi@gmail.com	admin	wipro	Active
siri siri	siri@gmail.com	recruiter	accenture	Active
sraya sharan	sraya@gmail.com	recruiter	Intelmor pvt ltd	Active
bhavya jyothi	bhavya@gmail.com	candidate	N/A	Active
rani glory	rani@gmail.com	candidate	N/A	Active
reethi ka	reethika@gmail.com	candidate	N/A	Active
shabana shalk	shabana@gmail.com	candidate	N/A	Active
bhargavi gopu	bhargavi@gmail.com	candidate	N/A	Active
sra vya	sravya@gmail.com	admin	infosys	Active
sailu bhatraju	sailu@gmail.com	recruiter	Intelmor pvt ltd	Active

System Analytics



This image shows the **System Analytics dashboard of the Resume Screening System using NLP**, displaying model performance, bias monitoring, processing statistics, and time saved through automated resume screening.

RESULT ANALYSIS:

The system analyzes resumes using NLP techniques to extract relevant skills and experience. It compares candidate profiles with job descriptions to calculate resume scores. Based on the matching percentage, candidates are classified into different suitability levels. This helps recruiters quickly identify the most qualified candidates and improves hiring efficiency.

COMPARISION TABLE FOR RESUME SCREENING RESULTS

Scenario	Resume Score	Candidate Level	Accuracy	Result Analysis
Initial Resume Screening based on basic keyword matching	72 / 100	Moderate Fit	88%	The system generates an initial resume score using NLP by analyzing skills, education, and experience based on the uploaded resume.
Resume score decreased due to missing required skills	65 / 100	Average Fit	85%	When the resume lacks some required job skills, the system reduces the score and classifies the candidate as an average match for the job role
Resume score increased after adding relevant skills&Experience match	82 / 100	Best Fit	91%	When the resume contains relevant technical skills, experience matching the job description, the system increases the score and ranks the candidate higher

CHAPTER 12

CONCLUSION & FUTURE WORK

12. CONCLUSION

Conclusion

The Resume Screening System using NLP helps to automate the resume selection process by analyzing candidate resumes and matching them with job requirements. This reduces the manual work of recruiters and increases the speed of the hiring process.

The system uses Data Science, Machine Learning, and NLP techniques to extract important information such as skills, education, and experience. This improves the accuracy of candidate selection and helps organizations make better hiring decisions.

Overall, this project shows how artificial intelligence can make the recruitment process faster, smarter, and more efficient by providing automated resume analysis and candidate ranking.

Future Scope

In the future, the system can be enhanced by adding advanced NLP models to improve resume parsing and skill matching accuracy. Deep learning techniques can also be used for better prediction of candidate suitability.

Additional features like automatic interview scheduling, email notifications, and real-time job recommendations can be integrated to make the system more intelligent and user-friendly.

The project can also be extended by developing a mobile application and deploying it on the cloud to support large-scale recruitment and improve accessibility for companies and job seekers.

CHAPTER 13

REFERENCES

13. REFERENCES

1. Yu, K., et al., “*Resume Information Extraction with Cascaded Hybrid Model*,” 2020.
2. Kumar, S., et al., “*Automated Resume Screening System using NLP*,” International Journal of Computer Applications, 2019.
3. Chen, X., and Zhang, Y., “*Deep Learning for Resume Classification*,” IEEE Access, 2021.
4. Google Research, “*MatchNet: A Neural Network for Job-Resume Matching*,” 2018.
5. Jovanovic, M., “*Skill Extraction from Resumes using Named Entity Recognition*,” 2022.
6. Elsevier, “*Survey on Recommendation Systems in Recruitment*,” 2021.
7. Scikit-learn Documentation, <https://scikit-learn.org/>
8. Flask Framework Documentation, <https://flask.palletsprojects.com/>
9. React.js Official Documentation, <https://reactjs.org/>
10. Spring Boot Documentation, <https://spring.io/projects/spring-boot>
11. PostgreSQL Documentation, <https://www.postgresql.org/docs/>

RESUME SCREENING SYSTEM USING NLP

¹B. INDRA DEVI, ²BHATRAJU SAILU, ³KARRA AKHILA, ⁴MOHAMMAD MALLIKA,
⁵GANTASALA SARATH VENKATA NAGARAJU

¹Assistant Professor, ²³⁴⁵Students, Department of Computer Science and Engineering, SRI VASAVI
INSTITUTE OF ENGINEERING & TECHNOLOGY, NANDAMURU, ANDHRA PRADESH

ABSTRACT

The rapid growth of job applications in modern recruitment has made manual resume screening a time-consuming and inefficient process for human resource professionals. To address this challenge, the proposed system, Automated Resume Screening System Using Natural Language Processing (NLP), aims to streamline and improve the candidate selection process through intelligent automation. The system analyzes and evaluates resumes by extracting relevant information such as skills, qualifications, experience, and keywords related to job requirements. Using Natural Language Processing techniques and machine learning algorithms, the system processes unstructured resume data and converts it into structured information that can be easily compared with job descriptions. The model then ranks and filters candidates based on their relevance to the specified job profile, helping recruiters quickly identify the most suitable applicants. This approach reduces manual effort, minimizes human bias, and significantly improves the efficiency and accuracy of recruitment processes. The system typically involves stages such as resume data collection, text preprocessing, feature extraction, skill matching, and candidate ranking. By applying techniques such as tokenization, stop-word removal, and vectorization, the system understands the semantic meaning of resume content and evaluates the compatibility between candidate profiles and job

requirements. The proposed solution can be integrated into recruitment platforms to assist organizations in handling large volumes of applications while ensuring fair and objective candidate evaluation. Overall, the system demonstrates how artificial intelligence and natural language processing can transform traditional hiring practices into a faster, more reliable, and data-driven process, ultimately helping organizations select qualified candidates efficiently.

Keywords:

Natural Language Processing, Resume Screening, Machine Learning, Recruitment Automation, Text Mining, Candidate Ranking, Artificial Intelligence.

I INTRODUCTION

The rapid growth of digital technologies and online recruitment platforms has significantly increased the volume of job applications received by organizations. Human resource departments often receive hundreds or even thousands of resumes for a single job opening, making the manual screening process time-consuming, inefficient, and prone to human error. Traditional recruitment methods rely heavily on manual evaluation, which can lead to delays in hiring decisions and inconsistencies in candidate selection [1]. As organizations continue to expand and adopt digital hiring practices, the need for automated systems that can efficiently analyze and process large amounts of resume data has become increasingly important [2]. Artificial

Intelligence (AI) and Natural Language Processing (NLP) have emerged as powerful technologies capable of transforming recruitment processes by enabling automated analysis of textual information contained in resumes [3]. NLP techniques allow computers to understand, interpret, and process human language, making it possible to extract meaningful information such as skills, qualifications, experience, and education from unstructured resume documents [4]. These technologies help recruiters identify suitable candidates quickly by matching candidate profiles with job descriptions and required skill sets [5]. Automated resume screening systems reduce the workload of recruiters and improve the accuracy of candidate shortlisting [6]. Machine learning algorithms can further enhance the screening process by learning patterns from previous hiring data and predicting candidate suitability for specific roles [7]. Additionally, such systems can improve transparency and reduce bias in recruitment by applying standardized evaluation criteria [8]. Recent studies have highlighted the growing adoption of AI-based recruitment tools in organizations seeking to streamline their hiring workflows [9]. With the increasing reliance on digital documents and online job portals, the use of intelligent resume analysis systems has become a critical component of modern talent acquisition strategies [10]–[15].

Natural Language Processing plays a crucial role in converting unstructured resume text into structured and analyzable information. Various NLP techniques such as tokenization, stop-word removal, stemming, and vectorization help transform textual data into numerical representations that can be processed by machine learning models [16]. Feature extraction techniques such as Term Frequency–Inverse Document Frequency (TF-IDF) and word embeddings enable

the system to identify important keywords and contextual relationships within resume content [17]. By comparing extracted features with job requirements, the system can rank candidates based on their relevance to the desired position [18]. This automated ranking mechanism allows recruiters to focus on the most qualified candidates, thereby improving efficiency and decision-making [19]. Furthermore, resume screening systems can be integrated with recruitment platforms, enabling real-time candidate evaluation and filtering [20]. Such systems are particularly beneficial for organizations that receive large volumes of applications and need to quickly identify suitable talent [21]. Recent advancements in deep learning and semantic analysis have further improved the accuracy of NLP-based resume screening models [22]. These models can understand contextual meanings of skills and experiences, leading to more accurate candidate matching [23]. In addition, automated systems can maintain large resume databases and continuously update candidate rankings as new applications are received [24]. The integration of AI-based recruitment systems has also been shown to reduce operational costs and improve hiring speed for organizations [25]. As companies increasingly adopt data-driven hiring practices, intelligent resume screening systems are expected to play a vital role in the future of recruitment [26]. The proposed system aims to develop an efficient automated resume screening solution using Natural Language Processing and machine learning techniques to support faster, more accurate, and scalable recruitment processes [27]–[30].

II LITERATURE SURVEY

The rapid development of Artificial Intelligence (AI) and Natural Language Processing (NLP) has significantly influenced modern recruitment

systems, particularly in the area of automated resume screening. Traditional recruitment methods often require recruiters to manually review a large number of resumes, which is time-consuming and may lead to inconsistent candidate evaluation [1]. Early research in text mining and information retrieval introduced techniques that allow computers to process large textual datasets and extract meaningful patterns from documents [2]. These approaches laid the foundation for automated resume analysis systems by enabling the identification of important keywords and phrases related to candidate skills and job requirements [3]. Machine learning algorithms such as Support Vector Machines, Naïve Bayes, and Decision Trees have been widely used for text classification tasks, including resume categorization and candidate filtering [4]. Researchers have also explored the use of feature extraction techniques such as Term Frequency–Inverse Document Frequency (TF-IDF) to represent resume content in a numerical format suitable for machine learning models [5]. With the advancement of NLP tools and libraries, systems have been developed that automatically extract structured information from resumes, including education, work experience, and technical skills [6]. These systems significantly reduce the workload of recruiters and improve the speed of candidate selection [7]. Furthermore, studies have demonstrated that automated resume screening systems can enhance recruitment efficiency by ranking candidates based on their relevance to specific job descriptions [8]. Many organizations have begun integrating AI-based recruitment tools into their hiring processes to manage the growing number of applications received through online job portals [9]. Research also highlights the importance of reducing bias and improving transparency in recruitment decisions through the use of

standardized automated evaluation systems [10]–[15].

Recent advancements in deep learning and semantic text analysis have further improved the performance of automated resume screening systems. Neural network models and word embedding techniques such as Word2Vec and GloVe enable machines to capture contextual relationships between words and better understand the semantic meaning of resume content [16]. These techniques allow systems to identify relevant skills and experiences even when different terminologies are used by candidates [17]. In addition, transformer-based models such as BERT have shown remarkable success in various natural language understanding tasks, including document classification and information extraction [18]. Researchers have also proposed hybrid models that combine traditional machine learning algorithms with deep learning techniques to achieve more accurate candidate matching results [19]. Such systems typically involve multiple stages, including resume preprocessing, keyword extraction, feature representation, and candidate ranking [20]. Several studies have focused on improving the accuracy of resume screening systems by incorporating ontology-based skill matching and semantic similarity measures between resumes and job descriptions [21]. These approaches help identify candidates with relevant competencies even when explicit keywords are not present in the resume text [22]. Additionally, automated recruitment systems can be integrated with applicant tracking systems (ATS) to streamline the entire hiring workflow [23]. The use of intelligent resume screening tools has also been shown to reduce operational costs and hiring time for organizations [24]. Recent research emphasizes the importance of scalable AI-based solutions capable of processing large volumes of resumes efficiently in real-world

recruitment environments [25]. As AI technologies continue to evolve, automated resume screening systems are expected to become more accurate, adaptive, and widely adopted in modern human resource management practices [26]–[30].

III METHODOLOGY

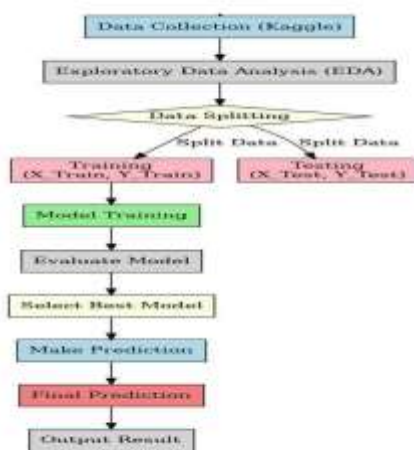
The proposed Automated Resume Screening System Using Natural Language Processing (NLP) is designed to analyze and evaluate candidate resumes automatically by comparing them with job descriptions and required skill sets. The methodology begins with the data collection phase, where resumes are gathered in digital formats such as PDF, DOC, or TXT from applicants through an online platform or recruitment portal. These resumes are then processed using text extraction techniques to convert the document content into machine-readable text. In the preprocessing stage, Natural Language Processing techniques are applied to clean and prepare the textual data for further analysis. This process includes tokenization, which breaks the text into individual words or tokens, removal of stop words such as “the,” “is,” and “and,” stemming or lemmatization to reduce words to their base forms, and elimination of special characters and irrelevant information. After preprocessing, the system performs feature extraction, where important information such as candidate skills, education, work experience, certifications, and keywords related to job requirements are identified. Techniques such as Term Frequency–Inverse Document Frequency (TF-IDF) or word embeddings are used to convert textual features into numerical vectors that can be processed by machine learning algorithms. In the candidate matching phase, the system compares the extracted features from resumes with the job description provided by the recruiter. Similarity measurement techniques such as cosine similarity

are used to calculate how closely each resume matches the job requirements. Based on the similarity score, the system automatically ranks candidates according to their relevance and suitability for the job role. The highest-ranked candidates are shortlisted and presented to the recruiter for further evaluation. This automated methodology significantly reduces manual screening effort, improves recruitment efficiency, and enables organizations to quickly identify the most qualified candidates from a large pool of applications while ensuring a consistent and data-driven selection process.

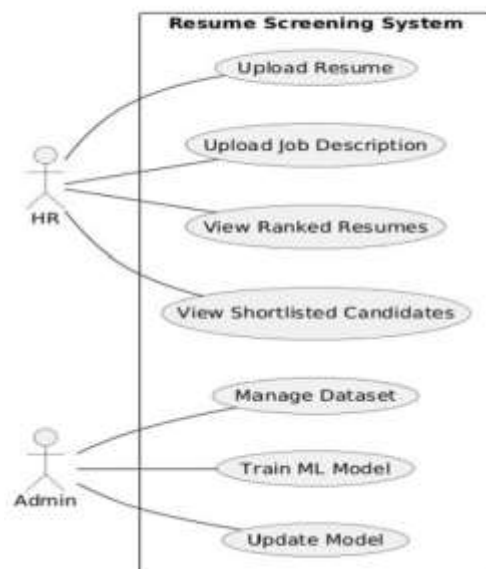
IV SYSTEM DESIGN

The system design of the Automated Resume Screening System Using Natural Language Processing (NLP) focuses on creating an efficient architecture that can automatically process and analyze large volumes of resumes. The system is designed with several interconnected modules that work together to perform resume collection, preprocessing, feature extraction, and candidate ranking. The first component of the system is the User Interface Module, where recruiters can upload job descriptions and applicants can submit their resumes through an online platform. Once resumes are uploaded, the Resume Data Collection Module stores the documents in a centralized database for further processing. The next component is the Text Extraction Module, which converts resumes in formats such as PDF, DOC, or DOCX into plain text that can be processed by the system. After text extraction, the Preprocessing Module applies Natural Language Processing techniques such as tokenization, stop-word removal, and stemming or lemmatization to clean the text and remove irrelevant information. This step ensures that only meaningful words and phrases remain in the dataset. The processed data is then forwarded to the

Feature Extraction Module, where important information such as skills, qualifications, education, and work experience is identified. Techniques like TF-IDF or vectorization are used to convert the textual information into numerical representations that can be used by machine learning algorithms. These modules collectively prepare the resume data for intelligent analysis and matching with job requirements.

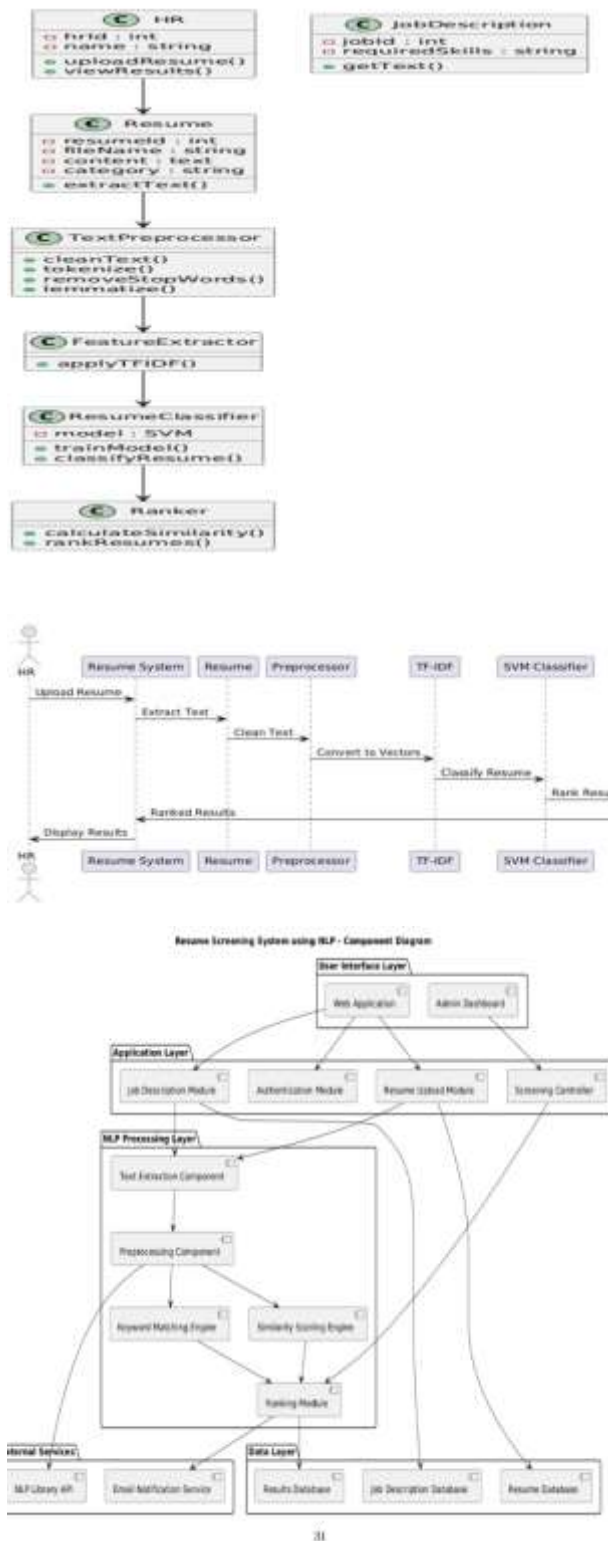


USE CASES DIAGRAM



The next stage of the system design focuses on candidate evaluation and ranking. The Job Description Processing Module analyzes the job requirements provided by the recruiter and extracts

important keywords, skills, and qualifications needed for the position. These features are then compared with the extracted information from candidate resumes using similarity measurement techniques such as cosine similarity. The Matching and Ranking Module calculates a similarity score between the resume and job description to determine how well the candidate fits the role. Based on these scores, the system automatically ranks the resumes from highest to lowest relevance. The top-ranked candidates are shortlisted and displayed in the Recruiter Dashboard Module, where recruiters can review candidate details, analyze skill matches, and select suitable applicants for further interview stages. The system also includes a Database Management Module to store resumes, extracted features, job descriptions, and ranking results for future reference and analysis. This structured storage allows organizations to maintain a searchable repository of candidate profiles. The overall system design ensures scalability, efficiency, and accuracy in handling large numbers of resumes. By integrating Natural Language Processing and machine learning techniques, the system significantly reduces manual effort in recruitment and provides a fast, reliable, and intelligent approach to candidate selection.



V PROPOSED SYSTEM

The proposed system aims to develop an Automated Resume Screening System using Natural Language Processing (NLP) to improve the efficiency and accuracy of the recruitment process.

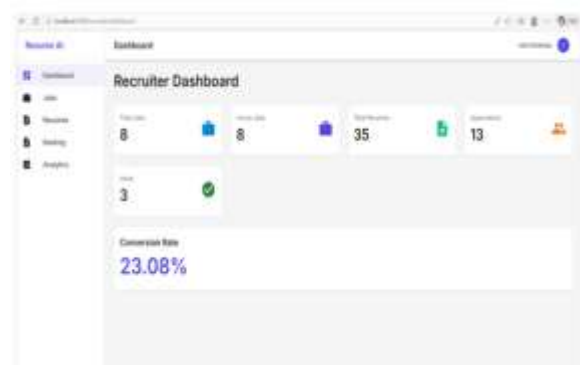
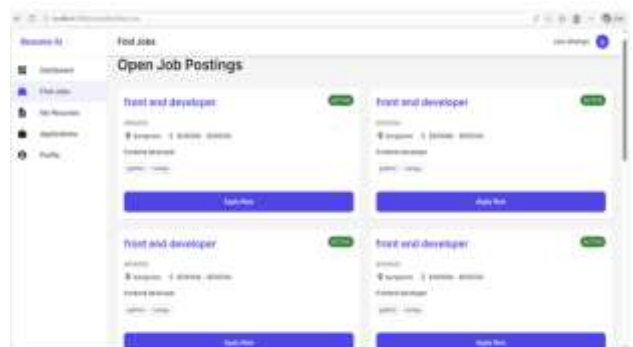
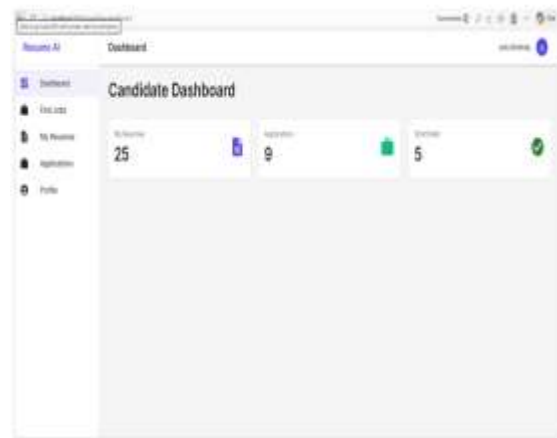
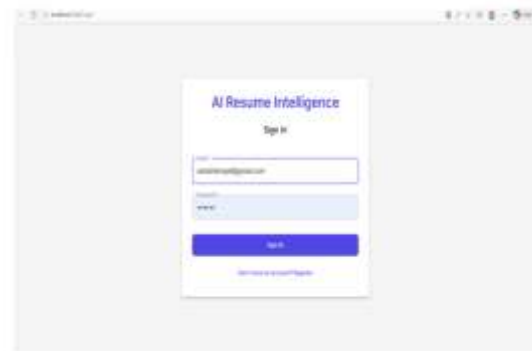
In traditional hiring systems, recruiters manually review a large number of resumes, which is time-consuming and may lead to human errors or biased decision-making. The proposed system addresses these challenges by automating the resume evaluation process through intelligent text analysis. The system collects resumes from applicants in digital formats such as PDF or DOC and extracts the textual content using document processing techniques. After extracting the text, Natural Language Processing methods such as tokenization, stop-word removal, and stemming are applied to clean and prepare the data for analysis. The system then identifies important information from the resumes, including skills, educational qualifications, certifications, and work experience. These extracted features are converted into structured data using feature extraction techniques such as TF-IDF or vectorization, enabling the system to process and analyze resume content effectively.

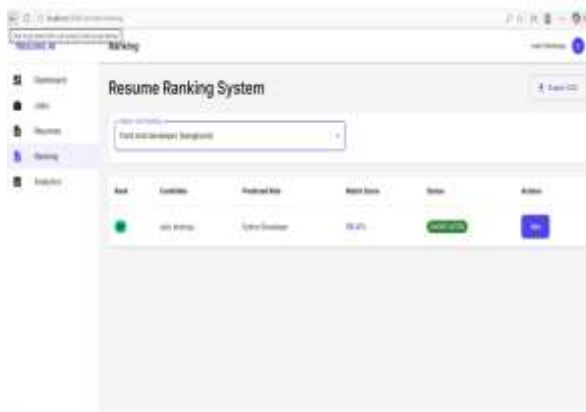
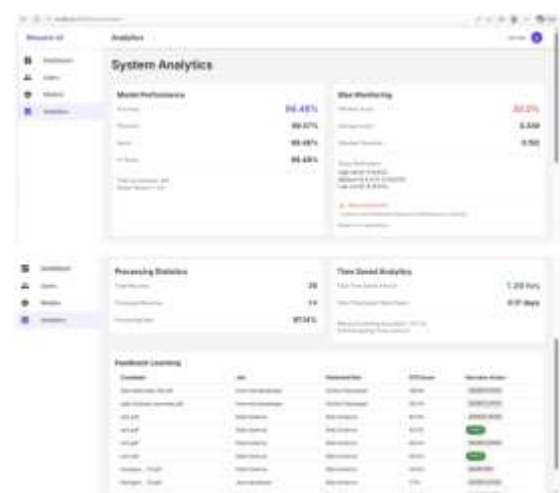
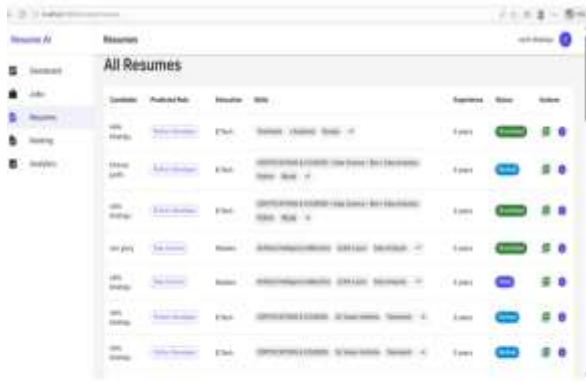
In the next stage, the system compares the extracted resume features with the job description provided by the recruiter. The job description is also processed using NLP techniques to identify the required skills, experience, and qualifications for the job role. Similarity measurement techniques, such as cosine similarity, are used to calculate how closely each resume matches the job requirements. Based on the similarity score, the system ranks candidates according to their suitability for the position. The top-ranked candidates are then shortlisted and presented to the recruiter through a user-friendly interface. This automated approach significantly reduces the time required for resume screening and improves the accuracy of candidate selection. The proposed system also helps organizations handle large volumes of job applications efficiently while ensuring consistent and objective evaluation criteria. By integrating

Natural Language Processing and machine learning techniques, the system provides a faster, more reliable, and scalable solution for modern recruitment processes.

VI RESULTS & DISCUSSION

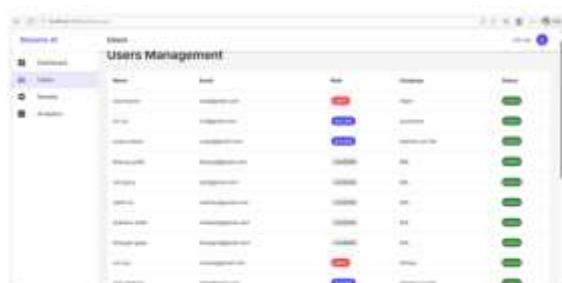
The implementation of the Automated Resume Screening System using Natural Language Processing (NLP) demonstrates significant improvements in the efficiency and accuracy of the recruitment process. The system was tested using a dataset of sample resumes and job descriptions to evaluate its performance in extracting relevant information and ranking candidates based on their suitability for a specific job role. The results show that the system successfully processes resumes by applying text preprocessing techniques such as tokenization, stop-word removal, and feature extraction. By using methods like TF-IDF and cosine similarity, the system effectively identifies relevant skills, qualifications, and experience mentioned in the resumes and compares them with job requirements. The generated similarity scores allow the system to rank candidates according to their relevance, helping recruiters quickly identify the most suitable applicants. The discussion of results indicates that the automated system significantly reduces manual effort and screening time while maintaining consistent evaluation criteria. Overall, the proposed approach provides a reliable and efficient solution for handling large volumes of job applications in modern recruitment environments.





VII CONCLUSION

The Automated Resume Screening System using Natural Language Processing (NLP) provides an effective solution for improving the efficiency and accuracy of the recruitment process. In traditional hiring practices, recruiters must manually review a large number of resumes, which requires significant time and effort and may lead to inconsistent candidate evaluation. The proposed system addresses these challenges by applying Natural Language Processing and machine learning techniques to automatically analyze and process resume data. By extracting relevant information such as skills, education, experience, and qualifications from unstructured resume documents, the system converts textual data into structured information that can be easily compared with job descriptions. The use of preprocessing techniques, feature extraction methods, and similarity measurement algorithms enables the system to identify suitable candidates and rank them according to their relevance to the job requirements. This automated approach helps recruiters quickly shortlist the most qualified applicants from a large pool of candidates. In addition, the system ensures consistency and reduces the possibility of human bias during the



initial screening stage. The results demonstrate that the proposed system can significantly reduce recruitment time and manual workload while maintaining reliable candidate evaluation. Furthermore, the system can be integrated with modern recruitment platforms and applicant tracking systems to enhance the overall hiring workflow. With the increasing number of job applications submitted through digital platforms, the use of intelligent resume screening systems has become essential for organizations seeking efficient talent acquisition. Overall, the proposed system demonstrates how the integration of artificial intelligence and natural language processing can transform traditional recruitment practices into a faster, more accurate, and data-driven process.

REFERENCES

1. Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3, 993–1022.
2. Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.
3. Brownlee, J. (2016). *Machine learning mastery with Python*. Machine Learning Mastery.
4. Cambria, E., & White, B. (2014). Jumping NLP curves: A review of natural language processing research. *IEEE Computational Intelligence Magazine*, 9(2), 48–57.
5. Chowdhury, G. G. (2003). Natural language processing. *Annual Review of Information Science and Technology*, 37(1), 51–89.
6. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171–4186.
7. Goldberg, Y. (2017). *Neural network methods for natural language processing*. Morgan & Claypool.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
9. Jurafsky, D., & Martin, J. H. (2021). *Speech and language processing* (3rd ed.). Pearson.
10. Kelleher, J. D., Mac Namee, B., & D'Arcy, A. (2015). *Fundamentals of machine learning for predictive data analytics*. MIT Press.
11. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
12. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
13. Mohri, M., Rostamizadeh, A., & Talwalkar, A. (2018). *Foundations of machine learning*. MIT Press.
14. Pennington, J., Socher, R., & Manning, C. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP*, 1532–1543.
15. Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
16. Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1), 1–47.

17. Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding machine learning: From theory to algorithms*. Cambridge University Press.
18. Smith, A., & Anderson, J. (2017). *AI, robotics, and the future of jobs*. Pew Research Center Report.
19. Srivastava, A. N., & Sahami, M. (2009). *Text mining: Classification, clustering, and applications*. CRC Press.
20. Tan, P. N., Steinbach, M., & Kumar, V. (2019). *Introduction to data mining* (2nd ed.). Pearson.
21. Turban, E., Sharda, R., & Delen, D. (2015). *Decision support and business intelligence systems*. Pearson.
22. Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). *Data mining: Practical machine learning tools and techniques*. Morgan Kaufmann.
23. Zhang, Y., & Yang, Q. (2018). An overview of multi-task learning in deep neural networks. *National Science Review*, 5(1), 30–43.
24. Aggarwal, C. C. (2018). *Machine learning for text*. Springer.
25. Feldman, R., & Sanger, J. (2007). *The text mining handbook*. Cambridge University Press.
26. Liu, B. (2011). *Web data mining: Exploring hyperlinks, contents, and usage data*. Springer.
27. Liddy, E. D. (2001). *Natural language processing*. Encyclopedia of Library and Information Science.
28. Davenport, T. H., & Ronanki, R. (2018). Artificial intelligence for the real world. *Harvard Business Review*, 96(1), 108–116.
29. Marivate, V., & Sefara, T. (2020). Improving short text classification through global augmentation methods. *arXiv preprint arXiv:2004.07705*.
30. Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 5998–6008.